



Universidad Autónoma de San Luis Potosí

Facultad de Ingeniería



Área de Ciencias de la Computación

Proyecto:

Servicios De Mantenimiento

Manual del Programador

Nombre de los alumnos:

- González Medina Claudia Karina
- Hernández López Eduardo Alejandro
- Piña Silva Juana María

Materia: Administración de Bases de Datos

Nombre del profesor: M.E.M. Claudia Alicia Méndez Hernández

Número de Grupo: 231301

Nombre del Semestre: 2024-2025/I

Horario: 13:00 a 14:00 hrs

Introducción

Para comenzar con el proyecto de Servicios de Mantenimiento en Postgres y Apache NetBeans debemos tener el conocimiento previo sobre el contenido de esta aplicación donde se realizarán las operaciones necesarias del CRUD para el manejo de una base de datos que tiene como nombre Servicios de Mantenimiento el cual pretende ofrecer servicios para el hogar, tales como servicio de electricidad, plomería, y reconstrucción.

Herramientas Utilizadas

Por lo cual en este proyecto es necesario implementar un script para el manejo y manipulación de la base de datos, que en este caso será en el sistema manejador de bases de datos conocido como PostgreSQL y Apache NetBeans IDE donde se implementará la interfaz posteriormente presentada en el proyecto.



Pgadmin 4: para la administración de la base de datos
"PostgreSQL 16.4, compiled by Visual C++ build 1940, 64-bit"

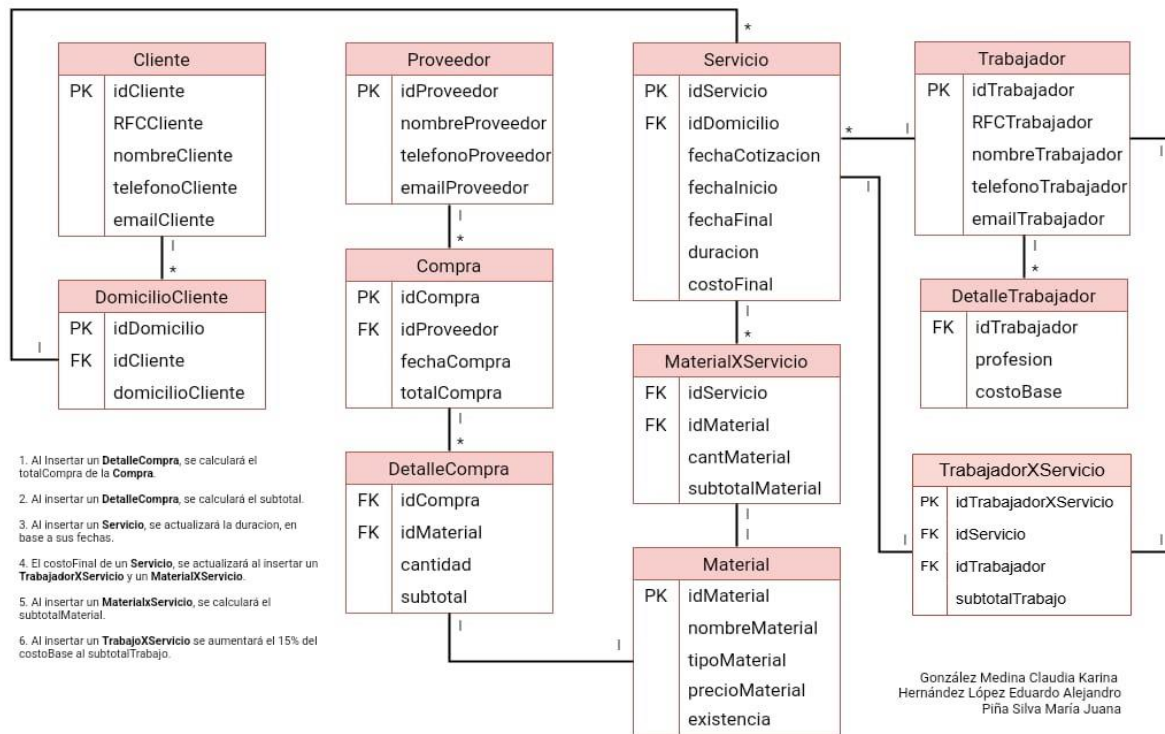


NetBeans: entorno para el desarrollo gráfico

Modelo Relación

En este proyecto se utilizó el siguiente diagrama para su desarrollo, donde cuenta con las tablas y relaciones necesarias para el correcto funcionamiento del proyecto.

Servicios de Mantenimiento



Script de la Base de Datos

El Modelo Relación que presentamos en un principio se implementará en la creación de la base de datos usando PostgreSQL.

```
-- Database: ServiciosDeMantenimiento

-- DROP DATABASE IF EXISTS "ServiciosDeMantenimiento";

CREATE DATABASE "ServiciosDeMantenimiento"
WITH
  OWNER = postgres
  ENCODING = 'UTF8'
  LC_COLLATE = 'Spanish_Mexico.1252'
  LC_CTYPE = 'Spanish_Mexico.1252'
  LOCALE_PROVIDER = 'libc'
  TABLESPACE = pg_default
  CONNECTION LIMIT = -1
  IS_TEMPLATE = False;

-- Crear esquemas
CREATE SCHEMA Comercial;
CREATE SCHEMA Operativo;

-- CLIENTE
CREATE TABLE Comercial.Cliente (
  idCliente BIGSERIAL PRIMARY KEY,
  RFCCliente VARCHAR(20) NOT NULL,
  nombreCliente VARCHAR(100) NOT NULL,
  telefonoCliente VARCHAR(10) NOT NULL,
  emailCliente VARCHAR(200) NOT NULL
);

-- DOMICILIOCLIENTE
CREATE TABLE Comercial.DomicilioCliente (
  idDomicilio BIGSERIAL PRIMARY KEY,
  idCliente BIGINT NOT NULL,
  domicilioCliente VARCHAR(200) NOT NULL,
  CONSTRAINT FK_CLIENTE01 FOREIGN KEY (idCliente) REFERENCES Comercial.Cliente (idCliente)
);

-- PROVEEDOR
CREATE TABLE Comercial.Proveedor (
  idProveedor BIGSERIAL PRIMARY KEY,
  nombreProveedor VARCHAR(200) NOT NULL,
  telefonoProveedor VARCHAR(10) NOT NULL,
  emailProveedor VARCHAR(200) NOT NULL,
  CONSTRAINT UQ_TELEFONOPROVEEDORUNICO UNIQUE (telefonoProveedor),
  CONSTRAINT UQ_EMAILPROVEEDORUNICO UNIQUE (emailProveedor)
);

-- COMPRA
CREATE TABLE Comercial.Compra (
  idCompra BIGSERIAL PRIMARY KEY,
  idProveedor BIGINT NOT NULL,
  fechaCompra DATE NOT NULL,
  totalCompra NUMERIC,
```

```
CONSTRAINT FK_PROVEEDOR01 FOREIGN KEY (idProveedor) REFERENCES Comercial.Proveedor
(idProveedor)
);
```

-- DETALLECOMPRA

```
CREATE TABLE Comercial.DetalleCompra (
    idCompra BIGINT NOT NULL,
    idMaterial BIGINT NOT NULL,
    cantidad INT NOT NULL,
    subtotal NUMERIC,
    CONSTRAINT FK_COMPRA FOREIGN KEY (idCompra) REFERENCES Comercial.Compra (idCompra),
    CONSTRAINT FK_MATERIAL FOREIGN KEY (idMaterial) REFERENCES Operativo.Material (idMaterial)
);
```

-- SERVICIO

```
CREATE TABLE Operativo.Servicio (
    idServicio BIGSERIAL PRIMARY KEY,
    idDomicilio BIGINT NOT NULL,
    fechaCotizacion DATE NOT NULL,
    fechaInicio DATE NOT NULL,
    fechaFinal DATE NOT NULL,
    duracion INT,
    costoFinal NUMERIC,
    CONSTRAINT FK_DOMICILIO FOREIGN KEY (idDomicilio) REFERENCES Comercial.DomicilioCliente
(idDomicilio)
);
```

-- MATERIAL

```
CREATE TABLE Operativo.Material (
    idMaterial BIGSERIAL PRIMARY KEY,
    nombreMaterial VARCHAR(100) NOT NULL,
    tipoMaterial VARCHAR(50) NOT NULL,
    precioMaterial NUMERIC NOT NULL,
    existencia INT NOT NULL,
    CONSTRAINT UQ_NOMBREMATERIALUNICO UNIQUE (nombreMaterial)
);
```

-- MATERIALXSERVICIO

```
CREATE TABLE Operativo.MaterialXServicio (
    idServicio BIGINT NOT NULL,
    idMaterial BIGINT NOT NULL,
    cantMaterial INT NOT NULL,
    subtotalMaterial NUMERIC,
    CONSTRAINT FK_SERVICIO FOREIGN KEY (idServicio) REFERENCES Operativo.Servicio (idServicio),
    CONSTRAINT FK_MATERIAL2 FOREIGN KEY (idMaterial) REFERENCES Operativo.Material (idMaterial)
);
```

-- TRABAJADOR

```
CREATE TABLE Operativo.Trabajador (
    idTrabajador BIGSERIAL PRIMARY KEY,
    RFCTrabajador VARCHAR(13) NOT NULL,
    nombreTrabajador VARCHAR(100) NOT NULL,
    telefonoTrabajador VARCHAR(10) NOT NULL,
    emailTrabajador VARCHAR(200) NOT NULL,
    CONSTRAINT UQ_RFCTRABAJADORUNICO UNIQUE (RFCTrabajador),
    CONSTRAINT UQ_TELEFONOTRABAJADORUNICO UNIQUE (telefonoTrabajador),
    CONSTRAINT UQ_EMAILTRABAJADORUNICO UNIQUE (emailTrabajador)
);
```

```

);

-- DETALLETRABAJADOR
CREATE TABLE Operativo.DetalleTrabajador (
    idTrabajador BIGINT NOT NULL,
    profesion VARCHAR(100) NOT NULL,
    costoBase NUMERIC NOT NULL,
    CONSTRAINT FK_TRABAJADOR FOREIGN KEY (idTrabajador) REFERENCES Operativo.Trabajador
(idTrabajador)
);

-- TRABAJADORXSERVICIO
CREATE TABLE Operativo.TrabajadorXServicio (
    idTrabajadorXServicio BIGSERIAL PRIMARY KEY,
    idServicio BIGINT NOT NULL,
    idTrabajador BIGINT NOT NULL,
    subtotalTrabajo NUMERIC(18, 2),
    CONSTRAINT FK_SERVICIO2 FOREIGN KEY (idServicio) REFERENCES Operativo.Servicio (idServicio),
    CONSTRAINT FK_TRABAJADOR2 FOREIGN KEY (idTrabajador) REFERENCES Operativo.Trabajador
(idTrabajador)
);

-- REGLAS CON CONSTRAINTS
ALTER TABLE Operativo.Material
ADD CONSTRAINT chk_tipo_material CHECK (tipoMaterial IN ('Plomeria', 'Reconstruccion', 'Electricidad'));

ALTER TABLE Comercial.DetalleCompra
ADD CONSTRAINT chk_cantidad_material CHECK (cantidad >= 1);

-- Disparadores para el proyecto
-- DISPARADOR(ES) PARA DETALLECOMPRA
-- 1. CALCULAR_SUBTOTAL_DETALLECOMPRA
-- ** Al insertar un DetalleCompra, se calculará el subtotal. ** --
CREATE OR REPLACE FUNCTION fn_CalcularSubtotalDetalleCompra()
RETURNS TRIGGER AS $$
DECLARE
    precio DECIMAL;
BEGIN
    -- Obtener el precio del material desde la tabla Operativo.Material
    SELECT precioMaterial
    INTO precio
    FROM Operativo.Material
    WHERE idMaterial = NEW.idMaterial;

    -- Calcular el subtotal
    NEW.subtotal := NEW.cantidad * precio;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TGR_CALCULAR_SUBTOTAL_DETALLECOMPRA
BEFORE INSERT OR UPDATE ON Comercial.DetalleCompra
FOR EACH ROW EXECUTE FUNCTION fn_CalcularSubtotalDetalleCompra();

-- 2. TGR_ACTUALIZATOTALCOMPRA
-- ** Al insertar un DetalleCompra, se calculará el totalCompra de la Compra. ** --
CREATE OR REPLACE FUNCTION fn_ActualizarTotalCompra()
RETURNS TRIGGER AS $$

```

```

BEGIN
    UPDATE Comercial.Compra
    SET totalCompra = ROUND((
        SELECT COALESCE(SUM(subtotal), 0::numeric)
        FROM Comercial.DetalleCompra
        WHERE idCompra = COALESCE(NEW.idCompra, OLD.idCompra)
    ), 2) -- Redondea a 2 decimales
    WHERE idCompra = COALESCE(NEW.idCompra, OLD.idCompra);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER TGR_ACTUALIZATOTALCOMPRA
AFTER INSERT OR UPDATE OR DELETE ON Comercial.DetalleCompra
FOR EACH ROW EXECUTE FUNCTION fn_ActualizarTotalCompra();

-- 3. TGR_ACTUALIZA_MATERIAL
-- ** Actualiza la existencia en Material al momento de insertar un DetalleCompra. ** --
CREATE OR REPLACE FUNCTION fn_ActualizarExistenciasMaterialenDC()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE Operativo.Material
        SET existencia = existencia + NEW.cantidad
        WHERE idMaterial = NEW.idMaterial;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE Operativo.Material
        SET existencia = existencia - OLD.cantidad + NEW.cantidad
        WHERE idMaterial = NEW.idMaterial;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE Operativo.Material
        SET existencia = existencia - OLD.cantidad
        WHERE idMaterial = OLD.idMaterial;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TGR_ACTUALIZA_MATERIAL
AFTER INSERT OR UPDATE OR DELETE ON Comercial.DetalleCompra
FOR EACH ROW EXECUTE FUNCTION fn_ActualizarExistenciasMaterialenDC();

--DISPARADOR(ES) PARA SERVICIO
-- 4. TGR_UPDATE_COSTO_FINAL
-- ** Al insertar un MaterialXServicio actualiza el costoFinal del Servicio. ** --
CREATE OR REPLACE FUNCTION fn_UpdateCostoFinalServicio()
RETURNS TRIGGER AS $$
DECLARE
    subtotalMaterialF DECIMAL;
    subtotalTrabajoF DECIMAL;
BEGIN
    -- Calculamos el subtotal de materiales
    SELECT COALESCE(SUM(subtotalMaterial), 0)
    INTO subtotalMaterialF
    FROM Operativo.MaterialXServicio
    WHERE idServicio = NEW.idServicio;

    -- Calculamos el subtotal de trabajo

```

```

SELECT COALESCE(SUM(subtotalTrabajo), 0)
INTO subtotalTrabajoF
FROM Operativo.TrabajadorXServicio
WHERE idServicio = NEW.idServicio;

-- Actualizamos el costoFinal solo si hay cambios
UPDATE Operativo.Servicio
SET costoFinal = ROUND(subtotalMaterialF + subtotalTrabajoF, 2)
WHERE idServicio = NEW.idServicio;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TGR_UPDATE_COSTO_FINAL_MATERIAL
AFTER INSERT OR UPDATE OR DELETE ON Operativo.MaterialXServicio
FOR EACH ROW EXECUTE FUNCTION fn_UpdateCostoFinalServicio();

CREATE TRIGGER TGR_UPDATE_COSTO_FINAL_TRABAJADOR
AFTER INSERT OR UPDATE OR DELETE ON Operativo.TrabajadorXServicio
FOR EACH ROW EXECUTE FUNCTION fn_UpdateCostoFinalServicio();

-- 5. TGR_DURACIONSERVICIO
-- ** Al insertar un Servicio se actualizará la duracion, en base a sus fechas. ** --
CREATE OR REPLACE FUNCTION fn_calcularduracionservicio()
RETURNS TRIGGER AS $$
BEGIN
    -- Verificar que los campos fechaInicio y fechaFinal existen
    IF NEW.fechaInicio IS NOT NULL AND NEW.fechaFinal IS NOT NULL THEN
        -- Calcular la duración en días entre las dos fechas
        NEW.duracion := 1 + (NEW.fechaFinal - NEW.fechaInicio); -- La duración es un valor en días
    ELSE
        -- Si alguna de las fechas es nula, asignar duración nula
        NEW.duracion := NULL;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TGR_DURACIONSERVICIO
BEFORE INSERT OR UPDATE ON Operativo.Servicio
FOR EACH ROW EXECUTE FUNCTION fn_CalcularDuracionServicio();

--DISPARADOR(ES) PARA MATERIAL
-- 6. TGR_CALCULA_SUBTOTAL_MATERIAL
-- ** Calcula el subtotalMaterial de un MaterialXServicio. ** --
CREATE OR REPLACE FUNCTION fn_CalcularSubtotalMaterial()
RETURNS TRIGGER AS $$
BEGIN
    NEW.subtotalMaterial := ROUND(
        NEW.cantMaterial *
        (SELECT precioMaterial FROM Operativo.Material WHERE idMaterial = NEW.idMaterial),
        2 -- Redondear a 2 decimales
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER TGR_CALCULA_SUBTOTAL_MATERIAL
BEFORE INSERT OR UPDATE ON Operativo.MaterialXServicio
FOR EACH ROW EXECUTE FUNCTION fn_CalculaSubtotalMaterial();

-- 7. TRG_UPDATECOSTOFINAL_MATERIALXSERVICIO
-- ** Al insertar un MaterialXServicio actualiza el costoFinal. ** --
CREATE OR REPLACE FUNCTION fn_UpdateCostoFinalMaterialXServicio()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT costoFinal FROM Operativo.Servicio WHERE idServicio = NEW.idServicio) IS NULL THEN
        UPDATE Operativo.Servicio
        SET costoFinal = ROUND(
            (SELECT COALESCE(SUM(subtotalMaterial), 0::numeric)
             FROM Operativo.MaterialXServicio
             WHERE idServicio = NEW.idServicio), 2) -- Redondear a 2 decimales
        WHERE idServicio = NEW.idServicio;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER TRG_UPDATECOSTOFINAL_MATERIALXSERVICIO
AFTER INSERT OR UPDATE ON Operativo.MaterialXServicio
FOR EACH ROW EXECUTE FUNCTION fn_UpdateCostoFinalMaterialXServicio();

```

```

-- 8. TGR_ACTUALIZAR_EXISTENCIAS_MATERIAL
-- ** Al insertar un MaterialXServicio se actualizará la existencia del Material. ** --
CREATE OR REPLACE FUNCTION fn_ActualizarExistenciasMaterial()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE Operativo.Material
        SET existencia = existencia - NEW.cantMaterial
        WHERE idMaterial = NEW.idMaterial;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE Operativo.Material
        SET existencia = existencia + OLD.cantMaterial - NEW.cantMaterial
        WHERE idMaterial = NEW.idMaterial;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE Operativo.Material
        SET existencia = existencia + OLD.cantMaterial
        WHERE idMaterial = OLD.idMaterial;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER TGR_ACTUALIZAR_EXISTENCIAS_MATERIAL
AFTER INSERT OR UPDATE OR DELETE ON Operativo.MaterialXServicio
FOR EACH ROW EXECUTE FUNCTION fn_ActualizarExistenciasMaterial();

```

```

--DISPARADOR(ES) PARA TRABAJADOR
-- 9. TGR_CALCULASUBTOTALTRABAJO
-- ** Al insertar un TrabajoXServicio se aumentará el 15% del costoBase al subtotalTrabajo. ** --
CREATE OR REPLACE FUNCTION fn_CalculaSubtotalTrabajo()
RETURNS TRIGGER AS $$
DECLARE

```

```

    totalCostoBase NUMERIC; -- Variable temporal para almacenar la suma de costoBase
BEGIN
    -- Calcular la suma de costoBase para el idTrabajador asociado
    SELECT SUM(costoBase)
    INTO totalCostoBase
    FROM Operativo.DetalleTrabajador
    WHERE idTrabajador = NEW.idTrabajador;

    -- Si no hay registros asociados, asignar subtotalTrabajo como 0
    IF totalCostoBase IS NULL THEN
        totalCostoBase := 0;
    END IF;

    -- Multiplicar por 1.15 y asignar a subtotalTrabajo
    NEW.subtotalTrabajo := totalCostoBase * 1.15;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER TGR_CALCULASUBTOTALTRABAJO
BEFORE INSERT OR UPDATE ON Operativo.TrabajadorXServicio
FOR EACH ROW EXECUTE FUNCTION fn_CalculaSubtotalTrabajo();

```

```

-- 10. TRG_UPDATECOSTOFINAL_TRABAJADORXSERVICIO
-- ** El costoFinal de un Servicio se actualizará al insertar un TrabajadorXServicio. ** --
CREATE OR REPLACE FUNCTION fn_UpdateCostoFinalTrabajadorXServicio()
RETURNS TRIGGER AS $$
BEGIN
    -- Actualizar el costoFinal después de un INSERT o UPDATE
    IF (TG_OP = 'INSERT' OR TG_OP = 'UPDATE') THEN
        UPDATE Operativo.Servicio
        SET costoFinal = (SELECT COALESCE(SUM(subtotalTrabajo), 0) FROM Operativo.TrabajadorXServicio
        WHERE idServicio = NEW.idServicio)
        WHERE idServicio = NEW.idServicio;

        -- Actualizar el costoFinal después de un DELETE
        ELSIF (TG_OP = 'DELETE') THEN
            UPDATE Operativo.Servicio
            SET costoFinal = (SELECT COALESCE(SUM(subtotalTrabajo), 0) FROM Operativo.TrabajadorXServicio
            WHERE idServicio = OLD.idServicio)
            WHERE idServicio = OLD.idServicio;
        END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER TRG_UPDATECOSTOFINAL_TRABAJADORXSERVICIO
AFTER INSERT OR UPDATE OR DELETE ON Operativo.TrabajadorXServicio
FOR EACH ROW EXECUTE FUNCTION fn_UpdateCostoFinalTrabajadorXServicio();

```

```

--USUARIOS
--USUARIO ADMINISTRADOR
CREATE USER admin WITH PASSWORD 'AdminPostgres';
GRANT ALL PRIVILEGES ON DATABASE "ServiciosDeMantenimiento" TO admin;

```

```
--USUARIO GERENTE
CREATE USER gerente WITH PASSWORD 'GerentePostgres';
GRANT CONNECT ON DATABASE "ServiciosDeMantenimiento" TO gerente;
GRANT USAGE ON SCHEMA Comercial, Operativo TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Comercial.Cliente TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Comercial.Proveedor TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Comercial.Compra TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Operativo.Trabajador TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Operativo.Material TO gerente;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE Operativo.Servicio TO gerente;

--USUARIO EMPLEADO
CREATE USER empleado WITH PASSWORD 'EmpleadoPostgres';
GRANT CONNECT ON DATABASE "ServiciosDeMantenimiento" TO empleado;
GRANT USAGE ON SCHEMA Comercial, Operativo TO empleado;
GRANT SELECT ON TABLE Comercial.Cliente TO gerente;
GRANT SELECT ON TABLE Comercial.Proveedor TO gerente;
GRANT SELECT ON TABLE Comercial.Compra TO gerente;
GRANT SELECT ON TABLE Operativo.Trabajador TO gerente;
GRANT SELECT ON TABLE Operativo.Material TO gerente;
GRANT SELECT ON TABLE Operativo.Servicio TO gerente;
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA Comercial, Operativo FROM empleado;
```

Conexión a la Base de Datos

En nuestro IDE Netbeans para realizar una correcta conexión primero se importará una dependencias. Luego de crear el proyecto, iremos a la carpeta "Project Files" y seleccionaremos el archivo pom.xml. Colocaremos el cursor después de las etiquetas "properties", las cuales debemos modificar para hacer la conexión a la base de datos en PostgreSQL. Se le agrego el siguiente:

```
<dependencies>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.4</version>
  </dependency>
</dependencies>
```

Clase Conexion

```
//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de
Mantenimiento.
package Tablas;
```

```
// Librerías del sistema usadas para crear y gestionar el formulario Servicio.
import java.sql.Connection;
import java.sql.DriverManager;
import javax.swing.JOptionPane;
```

```
/*Esta clase se utiliza solo para la coneccionn con la base de datos aqui
definimos las variables y le concatenamos estas variables a la cadena para completar la conexión*/
public class ConexionBD
{
    Connection conexion = null;
```

```

String usuario = "postgres";
String contraseña = "postgres";
String bd = "ServiciosDeMantenimiento";
String ip = "localhost";
String puerto = "5432";

String cadena = "jdbc:postgresql://" + ip + ":" + puerto + "/" + bd;

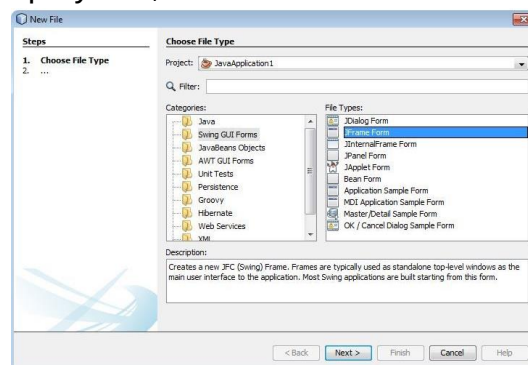
public Connection establecerConexion()
{
    try{
        Class.forName("org.postgresql.Driver");
        conexion = DriverManager.getConnection(cadena,usuario,contraseña);
        //JOptionPane.showMessageDialog(null, "Se conectó correctamente");
    }catch(Exception e){
        JOptionPane.showMessageDialog(null, "ERROR:" + e.toString());
    }
    return conexion;
}
}

```

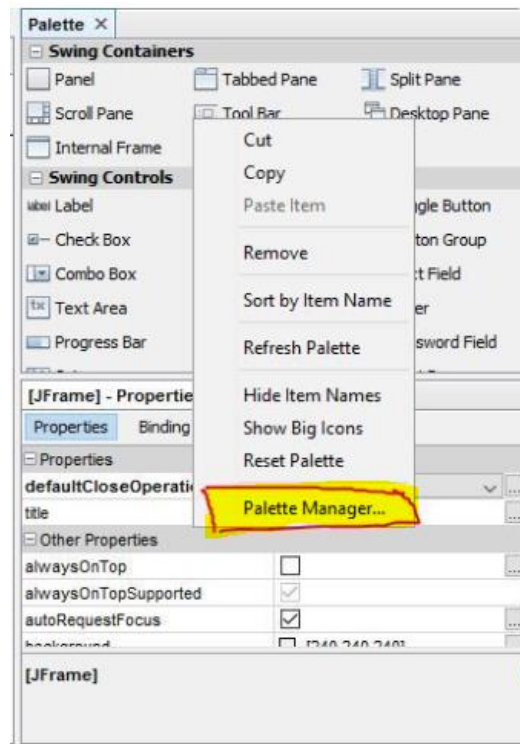
Importar JCalendar

Se usará una herramienta llamada Jcalendar que es un jar que contiene componentes gráficos para añadir a nuestra paleta de componentes. Este jar en concreto, nos añaden varios elementos relacionados con fechas.

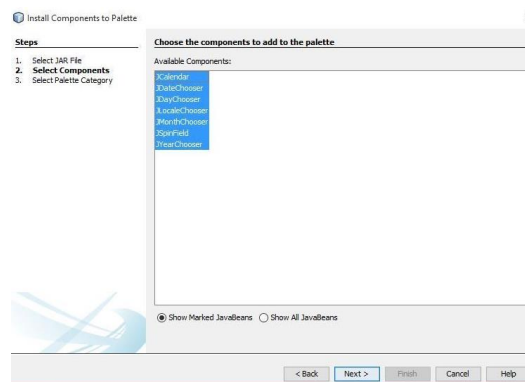
Lo primero en cualquier proyecto, crear un JFrame Form.



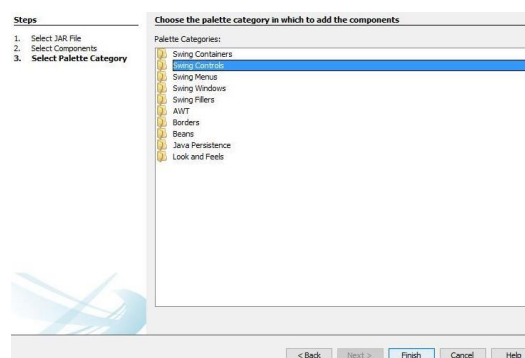
Después, estaremos en el modo diseño y en la paleta, damos click con el botón secundario, seleccionamos **Palette Manager**



Pulsamos en Add JAR, seleccionamos el JAR que hemos descargado y veremos esto:



Seleccionamos todos y pulsamos en siguiente.



Elegimos donde lo queremos guardar, os recomiendo en **Swing Controls**. Pulsamos en **Finish**. Ahora vemos esto en la paleta que son controles que podemos agregar a nuestro proyecto.



Servicio de Mantenimiento Usuarios

Se usará un login en el que se ingresaran usuarios con diferentes permisos, tenemos al usuario admin con todos los privilegios otorgados, al usuario gerente que puede realizar consultas, agregar y modificar en ciertas tablas como lo son la de Cliente, Proveedor, Compra, Trabajador, Material y Servicio. Por último, tenemos al usuario empleado que solo realizara consultas.

//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Usuarios.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
```

```
/*
```

Clase para gestionar el manejo de los usuarios para la gestión de los servicios dentro de los Servicios de Mantenimiento

Proporciona métodos para conexión, inicio de sesión, obtener el rol y abrir el menú.

```
*/
```

```
public class Usuarios extends javax.swing.JFrame
{
    private String usuario;
```

```

private String rol;

/*
    Constructor del formulario Usuarios.
    Inicializa el formulario.
*/
public Usuarios() {
    initComponents();
}

/*
    connectToDatabase
    Realiza la conexión con la base de datos.
*/
private Connection connectToDatabase(String username, String password)
{
    Connection connection = null;
    String url = "jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento"; //
URL de tu base de datos
    try {
        // Cargar el controlador JDBC
        Class.forName("org.postgresql.Driver");
        // Conectarse con las credenciales ingresadas
        connection = DriverManager.getConnection(url, username, password);
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Error al conectar a la base de datos:
" + e.getMessage());
    }
    return connection;
}

/*
    iniciarSesion
    Regresa el usuario al iniciar correctamente sesión con el usuario y contraseña
correcta de la base de datos.
*/
public String iniciarSesion()
{
    usuario = txtUsuario.getText(); // Campo de texto del usuario
    String contraseña = new String(txtContraseña.getPassword()); // Campo de
contraseña
    Connection bd = connectToDatabase(usuario, contraseña);
    if (bd != null) {
        rol = obtenerRolDeUsuario(usuario);
    }
}

```

```

        JOptionPane.showMessageDialog(this, "Inicio de sesión exitoso como: " +
usuario);
        abrirMenu(usuario, rol);
        try {
            bd.close(); // Cerrar la conexión después de usarla
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error al cerrar la conexión: " +
e.getMessage());
        }
    } else {
        JOptionPane.showMessageDialog(this, "Usuario o contraseña incorrectos
incorrectas.");
    }
    return usuario;
}

```

```

/*
    obtenerRolDeUsuario
    Obtiene el tipo de Rol del usuario.
*/
private String obtenerRolDeUsuario(String usuario)
{
    if (usuario.equals("gerente")) {
        return "Gerente";
    } else if (usuario.equals("empleado")) {
        return "Empleado";
    } else {
        return "Admin"; // Aquí se podría incluir otro tipo de rol
    }
}

```

```

/*
    abrirMenu
    Crea un nuevo formulario Menu.
*/
private void abrirMenu(String usuario, String rol)
{
    Menu menu = new Menu(usuario, rol);
    menu.setVisible(true);
    this.dispose();
}

```

```

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {

```

```

        public void run() {
            new Usuarios().setVisible(true);
        }
    });
}

```



//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

```

import javax.swing.JButton;
import javax.swing.JOptionPane;

```

```
/*
```

Clase principal del proyecto de Servicios de Mantenimiento.

Este formulario actúa como el menú principal de la aplicación, proporcionando acceso a otros formularios específicos.

Los siguiente formularios son:

- Formulario de Material.
- Formulario de Cliente.
- Formulario de Proveedor.
- Formulario de Trabajador.
- Formulario de Compra.
- Formulario de Servicio.

```
*/
```

```

public class Menu extends javax.swing.JFrame
{

```

```

    private String usuario;
    private String rol;

```

```
/*
```

Constructor del formulario principal.

Inicializa los componentes del formulario.

```
*/
```

```

public Menu(String usuario, String rol)

```

```

{
    this.usuario = usuario;
    this.rol = rol;
    initComponents();
    configurarPermisos();
}

/*
    Habilita y deshabilita los botones para el menú de cada tipo de usuario.
*/
public void configurarPermisos()
{
    switch (rol) {
        case "Admin":
            // Si es Admin, habilitar todo
            btnConsulta1.setVisible(true);
            btnConsulta2.setVisible(true);
            btnMaterial.setEnabled(true);
            btnCliente.setEnabled(true);
            btnProveedor.setEnabled(true);
            btnTrabajador.setEnabled(true);
            btnCompra.setEnabled(true);
            btnServicio.setEnabled(true);
            break;
        case "Gerente":
            // Si es Gerente, habilitar solo algunas opciones
            btnConsulta1.setVisible(false);
            btnConsulta2.setVisible(false);
            btnMaterial.setEnabled(true);
            btnCliente.setEnabled(true);
            btnProveedor.setEnabled(true);
            btnTrabajador.setEnabled(true);
            btnCompra.setEnabled(true);
            btnServicio.setEnabled(true);
            break;
        case "Empleado":
            // Si es Empleado, habilitar solo algunas opciones
            btnConsulta1.setVisible(false);
            btnConsulta2.setVisible(false);
            btnMaterial.setEnabled(true);
            btnCliente.setEnabled(true);
            btnProveedor.setEnabled(true);
            btnTrabajador.setEnabled(true);
            btnCompra.setEnabled(true);
    }
}

```

```

        btnServicio.setEnabled(true);
        break;
    default:
        break;
    }
}

/*
    btnMaterialActionPerformed
    Evento que se dispara al hacer clic en el botón de Material.
    Abre el formulario de Material como un cuadro de diálogo, permitiendo
gestionar los datos de materiales.
*/
private void btnMaterialActionPerformed(java.awt.event.ActionEvent evt) {
    Material materialForm = new Material(usuario, rol);
    materialForm.setVisible(true);
}

/*
    btnClienteActionPerformed
    Evento que se dispara al hacer clic en el botón de Cliente.
    Abre el formulario de Cliente como un cuadro de diálogo, permitiendo gestionar
los datos de los clientes.
*/
private void btnClienteActionPerformed(java.awt.event.ActionEvent evt) {
    Cliente clienteForm = new Cliente(usuario, rol);
    clienteForm.setVisible(true);
}

/*
    btnProveedorActionPerformed
    Evento que se dispara al hacer clic en el botón de Proveedor.
    Abre el formulario de Proveedor como un cuadro de diálogo, permitiendo
gestionar los datos de los proveedores.
*/
private void btnProveedorActionPerformed(java.awt.event.ActionEvent evt) {
    Proveedor proveedorForm = new Proveedor(usuario, rol);
    proveedorForm.setVisible(true);
}

/*
    btnTrabajadorActionPerformed
    Evento que se dispara al hacer clic en el botón de Trabajador.

```

Abre el formulario de Trabajador como un cuadro de diálogo, permitiendo gestionar los datos de los trabajadores.

```
*/
```

```
private void btnTrabajadorActionPerformed(java.awt.event.ActionEvent evt) {  
    Trabajador trabajadorForm = new Trabajador(usuario, rol);  
    trabajadorForm.setVisible(true);  
}
```

```
/*
```

btnCompraActionPerformed
Evento que se dispara al hacer clic en el botón de Compra.
Abre el formulario de Compra como un cuadro de diálogo, permitiendo gestionar los datos de las compras.

```
*/
```

```
private void btnCompraActionPerformed(java.awt.event.ActionEvent evt) {  
    Compra compraForm = new Compra(usuario, rol);  
    compraForm.setVisible(true);  
}
```

```
/*
```

btnServicioActionPerformed
Evento que se dispara al hacer clic en el botón de Servicio.
Abre el formulario de Servicio como un cuadro de diálogo, permitiendo gestionar los datos de los servicios.

```
*/
```

```
private void btnServicioActionPerformed(java.awt.event.ActionEvent evt) {  
    Servicio servicioForm = new Servicio(usuario, rol);  
    servicioForm.setVisible(true);  
}
```

```
/*
```

btnConsulta2ActionPerformed
Evento que se dispara al hacer clic en el botón de Consulta 1.
Abre el formulario de Consulta 1 como un cuadro de diálogo, permitiendo gestionar las consulta 1: Uso de GROUP BY, COUNT e INNER JOIN.

```
*/
```

```
private void btnConsulta1ActionPerformed(java.awt.event.ActionEvent evt) {  
    Consulta1 consulta1Form = new Consulta1();  
    consulta1Form.setVisible(true);  
}
```

```
/*
```

btnConsulta2ActionPerformed
Evento que se dispara al hacer clic en el botón de Consulta 2.

Abre el formulario de Consulta 2 como un cuadro de diálogo, permitiendo gestionar los consulta 2: Realizar subconsultas.

```

*/
private void btnConsulta2ActionPerformed(java.awt.event.ActionEvent evt) {
    Consulta2 consulta2Form = new Consulta2();
    consulta2Form.setVisible(true);
}

public static void main(String args[]) {
    String usuario = "";
    String rol = "";
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Menu(usuario, rol).setVisible(true);
        }
    });
}
}

```

Cliente

Id Cliente	RFC del Cliente	Nombre del Cli...	Teléfono del Cli...	Email del Cliente
2	3err4rwqr	jasidj	3434223411	fef@gmail.com
3	RUKJ241204TJ5	Jenni Ruby Kim	5677842123	jenRubkim@g...
4	AASH620524S...	Ashlin Hernand...	4467823292	hass88@gmail...
5	ZYFR790920G...	Zyan Lucas Mar...	894521244	zy@gmail.com
6	TOBL891220IH5	Tomas Blanco ...	6780909086	Tom@gmail.com

//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

package Tablas

// Librerías del sistema usadas para crear y gestionar el formulario Cliente

```

import java.util.List;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

```

```

/*
Clase para gestionar el manejo de los clientes dentro de los Servicios de
Mantenimiento
Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class Cliente extends javax.swing.JFrame
{
    int idCliente = -1;
    // Lista que almacena objetos de tipo DomicilioCliente.
    private List<DomicilioCliente> domicilioClienteForms = new
    ArrayList<DomicilioCliente>();

    /* Constructor del formulario Cliente.
    Inicializa el formulario y carga los datos de la tabla Cliente en el JTable.*/
    public Cliente()
    {
        initComponents();
        ConsultaDatos();
    }

    /*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Comercial.Cliente en el
    JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
    */
    private void ConsultaDatos()
    {

        String[] datos = new String[5];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Id Cliente",
        "RFC del Cliente", "Nombre del Cliente", "Teléfono del Cliente", "Email del
        Cliente"}, 0);
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;
        try
        {
            // Conexión a la base de datos de Postgres.
            bd =
            DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
            "postgres");

```

```

String sql = "SELECT * FROM Comercial.Cliente";
stmt = bd.prepareStatement(sql);
rs = stmt.executeQuery();
TablaCliente.setModel(model);

// Agrega los valores a una nueva fila para almacenarlos en el JTable.
while(rs.next()){
    datos[0] = String.valueOf(rs.getInt("idCliente"));
    datos[1] = rs.getString("RFCCliente");
    datos[2] = rs.getString("nombreCliente");
    datos[3] = rs.getString("telefonoCliente");
    datos[4] = rs.getString("emailCliente");
    model.addRow(datos);
}
rs.close();
stmt.close();
bd.close();
} catch(SQLException ex){
    JOptionPane.showMessageDialog(null, "Error al cargar el Cliente.");
}
}

/* InsertaDato
    Inserta un nuevo registro en la tabla Comercial.Cliente con los datos
    ingresados.
    Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un
    mensaje de error */
private void InsertaDato()
{
    String rfc = txtRFCCliente.getText();
    String nombre = txtNombreCliente.getText();
    String telefono = txtTelefonoCliente.getText();
    String email = txtEmailCliente.getText();

    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establece la conexión a la base de datos
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

```

```

// Verifica si el RFC, teléfono o correo ya existen
String verifica = "SELECT COUNT(*) FROM Comercial.Cliente WHERE
RFCCliente = ? OR telefonoCliente = ? OR emailCliente = ?";
stmt = bd.prepareStatement(verifica);
stmt.setString(1, rfc);
stmt.setString(2, telefono);
stmt.setString(3, email);

// Ejecuta la consulta
rs = stmt.executeQuery();
int count = 0;
if (rs.next()) {
    count = rs.getInt(1);
}

if (count > 0) {
    JOptionPane.showMessageDialog(null, "El RFC, el teléfono o el correo
electrónico ya están en uso.");

    txtRFCCliente.setText("");
    txtNombreCliente.setText("");
    txtTelefonoCliente.setText("");
    txtEmailCliente.setText("");
} else {
    // Inserta el nuevo cliente
    String sql = "INSERT INTO Comercial.Cliente (RFCCliente,
nombreCliente, telefonoCliente, emailCliente) VALUES (?, ?, ?, ?)";
    stmt.close();
    stmt = bd.prepareStatement(sql);
    stmt.setString(1, rfc);
    stmt.setString(2, nombre);
    stmt.setString(3, telefono);
    stmt.setString(4, email);

    stmt.executeUpdate();

    txtRFCCliente.setText("");
    txtNombreCliente.setText("");
    txtTelefonoCliente.setText("");
    txtEmailCliente.setText("");

    rs.close();
    stmt.close();
    bd.close();
}

```

```

    }
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al insertar el Cliente: " +
ex.getMessage());
}
}

```

ModificaDato

Modifica el registro seleccionado en la tabla Comercial.Cliente con los datos actualizados.

Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un mensaje de error.

```

private void ModificaDato(int idCliente)
{
    String rfc = txtRFCCliente.getText();
    String nombre = txtNombreCliente.getText();
    String telefono = txtTelefonoCliente.getText();
    String email = txtEmailCliente.getText();

    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try
    {
        // Establece la conexión a la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Verifica si el RFC, teléfono o correo ya existen
        String verifica = "SELECT COUNT(*) FROM Comercial.Cliente WHERE
RFCCliente = ? OR telefonoCliente = ? OR emailCliente = ?";
        stmt = bd.prepareStatement(verifica);
        stmt.setString(1, rfc);
        stmt.setString(2, telefono);
        stmt.setString(3, email);

        // Ejecuta la consulta
        rs = stmt.executeQuery();
        int count = 0;

```

```

        if (rs.next()) {
            count = rs.getInt(1);
        }

        if (count > 0) {
            JOptionPane.showMessageDialog(null, "El RFC, el teléfono o el correo electrónico ya están en uso.");

            txtRFCCliente.setText("");
            txtNombreCliente.setText("");
            txtTelefonoCliente.setText("");
            txtEmailCliente.setText("");
        } else {
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
            String sql = "UPDATE Comercial.Cliente SET RFCCliente = ?,
nombreCliente = ?, telefonoCliente = ?, emailCliente = ? WHERE idCliente = ?";
            stmt = bd.prepareStatement(sql);
            stmt.setString(1, rfc);
            stmt.setString(2, nombre);
            stmt.setString(3, telefono);
            stmt.setString(4, email);
            stmt.setInt(5, idCliente);

            stmt.executeUpdate();

            txtRFCCliente.setText("");
            txtNombreCliente.setText("");
            txtTelefonoCliente.setText("");
            txtEmailCliente.setText("");

            stmt.close();
            bd.close();
        }
    } catch (SQLException ex){
        JOptionPane.showMessageDialog(null, "Error al modificar el Cliente.");
    }
}

```

EliminaDato

Elimina el registro seleccionado en la tabla Comercial.Cliente con los datos actualizados.

```

private void EliminaDato(int idCliente)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try
    {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
        String sql = "DELETE FROM Comercial.Cliente WHERE idCliente = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idCliente);

        stmt.executeUpdate();

        txtRFCCliente.setText("");
        txtNombreCliente.setText("");
        txtTelefonoCliente.setText("");
        txtEmailCliente.setText("");

        stmt.close();
        bd.close();
    }catch(SQLException ex){
        JOptionPane.showMessageDialog(null, "Error al eliminar el Cliente.");
    }
}

```

SeleccionaDato

Selecciona los datos del cliente y los muestra en los campos de texto correspondientes.

```

private void SeleccionaDato(int idCliente)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try
    {

```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
        "postgres");
        String sql = "SELECT RFCCliente, nombreCliente, telefonoCliente,
emailCliente FROM Comercial.Cliente WHERE idCliente = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idCliente);
        rs = stmt.executeQuery();
        if (rs.next()) {
            txtRFCCliente.setText(rs.getString("RFCCliente"));
            txtNombreCliente.setText(rs.getString("nombreCliente"));
            txtTelefonoCliente.setText(rs.getString("telefonoCliente"));
            txtEmailCliente.setText(rs.getString("emailCliente"));
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch(SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al obtener los datos del
Cliente.");
        ex.printStackTrace();
    }
}

/*
    btnInsertarActionPerformed
    Evento de clic del botón de inserción.
    Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
*/
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    InsertaDato();
    ConsultaDatos();
}

/*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
ConsultaDatos.
*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaCliente.getSelectedRow();

```

```

        if(fila >= 0){
            int idCliente = Integer.parseInt(TablaCliente.getValueAt(fila, 0).toString());
            ModificaDato(idCliente);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
        btnEliminarActionPerformed
        Evento de clic del botón de eliminación.
        Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a
ConsultaDatos.
    */
    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaCliente.getSelectedRow();
        if(fila >= 0){
            int idCliente = Integer.parseInt(TablaCliente.getValueAt(fila, 0).toString());
            EliminaDato(idCliente);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
        TablaClienteMouseClicked
        Evento que se activa al hacer clic en una celda del JTable.
        Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
    */
    private void TablaClienteMouseClicked(java.awt.event.MouseEvent evt) {
        int fila = TablaCliente.getSelectedRow();
        if(fila >= 0) {
            idCliente = Integer.parseInt(TablaCliente.getValueAt(fila, 0).toString());
            SeleccionaDato(idCliente);

            btnDomicilioCliente.setEnabled(true);
        }
    }

```

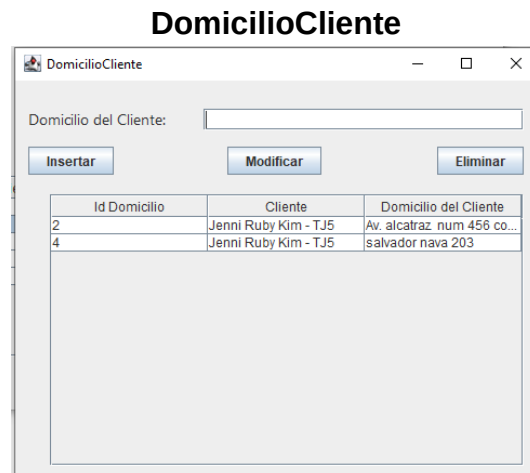
```

else {
    JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila válida.");
}
}

private void btnDomicilioClienteActionPerformed(java.awt.event.ActionEvent evt) {
    if(idCliente != -1)
    {
        DomicilioCliente domicilioClienteForm = new DomicilioCliente(idCliente);
        domicilioClienteForms.add(domicilioClienteForm);
        domicilioClienteForm.setVisible(true);
    }
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Cliente().setVisible(true);
        }
    });
}
}

```



// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Servicio.

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

```

```

import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/*
    Clase para gestionar el manejo de los detalles de los trabajadores dentro de los
    Servicios de Mantenimiento
    Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class DomicilioCliente extends javax.swing.JFrame
{
    int idCliente = -1;

    /*
        Constructor del formulario DetalleTrabajador.
        Inicializa el formulario y carga los datos de la tabla DetalleTrabajador en el
        JTable.
    */
    public DomicilioCliente(int idCliente)
    {
        initComponents();
        this.idCliente = idCliente;
        ConsultaDatos(idCliente);
    }

    /*
        ConsultaDatos
        Recupera y muestra todos los registros de la tabla Operativo.DetalleTrabajador
        en el JTable.
        Creación del modelo de la tabla agregando los encabezados de cada columna
        donde inicialmente no contiene filas.
    */
    private void ConsultaDatos(int idCliente)
    {
        String[] datos = new String[3];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Id Domicilio",
"Cliente", "Domicilio del Cliente"}, 0);
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

```

```

// Consulta con JOIN para obtener el nombre del trabajador
String sql = "SELECT d.idDomicilio, d.idCliente, d.domicilioCliente, " +
             "c.NombreCliente, c.RFCCliente " +
             "FROM Comercial.DomicilioCliente d " +
             "JOIN Comercial.Cliente c ON d.idCliente = c.idCliente " +
             "WHERE c.idCliente = ?";

stmt = bd.prepareStatement(sql);
stmt.setInt(1, idCliente);
rs = stmt.executeQuery();

TablaDomicilioCliente.setModel(model);

while (rs.next())
{
    String nombreCliente = rs.getString("NombreCliente");
    String rfcCliente = rs.getString("RFCCliente");
    String cliente = nombreCliente + " - " + rfcCliente.substring(rfcCliente.length()
- 3);

    datos[0] = rs.getString("idDomicilio");
    datos[1] = cliente;
    datos[2] = rs.getString("domicilioCliente");
    model.addRow(datos);
}
// Cerrar conexiones.
rs.close();
stmt.close();
bd.close();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar el Material del
Servicio: " + ex.getMessage());
}
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Operativo.DetalleTrabajador con los datos
ingresados.
Si ocurre un error(como duplicación de la profesión), muestra un mensaje de
error.
*/
private void InsertaDato(int idCliente)
{
    Connection bd = null;

```

```

PreparedStatement stmt = null;
ResultSet rs = null;

try {
    // Establecer conexión con la base de datos
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

    // Obtener datos del formulario
    String domicilio = txtDomicilio.getText();

    //Verifica si domicilio ya existe.
    String verificaProfesion = "SELECT COUNT(*) FROM
Comercial.DomicilioCliente WHERE idCliente = ? AND domicilioCliente = ?";
    stmt = bd.prepareStatement(verificaProfesion);
    stmt.setInt(1, idCliente);
    stmt.setString(2, domicilio);
    rs = stmt.executeQuery();

    if (rs.next() && rs.getInt(1) > 0) {
        JOptionPane.showMessageDialog(null, "El domicilio ya existe para este
cliente.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    String sql = "INSERT INTO Comercial.DomicilioCliente (idCliente,
domicilioCliente) VALUES (?, ?)";
    stmt = bd.prepareStatement(sql);
    stmt.setInt(1, idCliente);
    stmt.setString(2, domicilio);

    stmt.executeUpdate();

    // Limpia los campos después de insertar
    txtDomicilio.setText("");

} catch (SQLException ex) {
    JOptionPane.showMessageDialog(this, "Error al insertar el domicilio del
cliente: " + ex.getMessage());
} finally {
    try {
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    }
}

```

```

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Comercial.DomicilioCliente con los
datos actualizados.
Si ocurre un error (como duplicación de domicilio), muestra un mensaje de error.
*/
private void ModificaDato(int idCliente)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        int fila = TablaDomicilioCliente.getSelectedRow();
        if (fila < 0) {
            JOptionPane.showMessageDialog(null, "Seleccione un domicilio para
modificar.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Obtén el ID del domicilio desde la tabla
        int idDomicilio = Integer.parseInt(TablaDomicilioCliente.getValueAt(fila,
0).toString());

        // Obtén el nuevo domicilio desde el formulario
        String nuevoDomicilio = txtDomicilio.getText();

        // Verifica si el nuevo domicilio ya existe
        String verificaDomicilio = "SELECT COUNT(*) FROM
Comercial.DomicilioCliente WHERE idCliente = ? AND domicilioCliente = ?";
        stmt = bd.prepareStatement(verificaDomicilio);
        stmt.setInt(1, idCliente);
        stmt.setString(2, nuevoDomicilio);

```

```

rs = stmt.executeQuery();

if (rs.next() && rs.getInt(1) > 0) {
    JOptionPane.showMessageDialog(null, "El domicilio ya existe para este
cliente.", "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

// Actualiza el domicilio
String sql = "UPDATE Comercial.DomicilioCliente SET domicilioCliente = ?
WHERE idDomicilio = ?";
stmt = bd.prepareStatement(sql);
stmt.setString(1, nuevoDomicilio);
stmt.setInt(2, idDomicilio);
stmt.executeUpdate();

txtDomicilio.setText("");

} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al modificar el domicilio del
cliente: " + e.getMessage());
} finally {
    try {
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Comercial.DomicilioCliente.
*/
private void EliminaDato(int idCliente)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try {

```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        int fila = TablaDomicilioCliente.getSelectedRow();
        if (fila < 0) {
            JOptionPane.showMessageDialog(null, "Seleccione un domicilio para eliminar.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Obtén el ID del domicilio desde la tabla
        int idDomicilio = Integer.parseInt(TablaDomicilioCliente.getValueAt(fila, 0).toString());

        // Elimina el registro
        String sql = "DELETE FROM Comercial.DomicilioCliente WHERE idDomicilio = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idDomicilio);
        stmt.executeUpdate();

        txtDomicilio.setText("");

    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Existe al menos un servicio asociado a dicho domicilio.");
    } finally {
        try {
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
TablaClienteMouseClicked
Evento que se activa al hacer clic en una celda del JTable.
*/
private void TablaDomicilioClienteMouseClicked(java.awt.event.MouseEvent evt) {
    int fila = TablaDomicilioCliente.getSelectedRow();

```

```

        if (fila >= 0) {
            String domicilio = TablaDomicilioCliente.getValueAt(fila, 2).toString();
            txtDomicilio.setText(domicilio);

        } else {
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila.");
        }
    }

    /*
        btnEliminarActionPerformed
        Evento de clic del botón de eliminación.
        Extrae el idTrabajador, llama a EliminaDato, actualiza el JTable llamando a
        ConsultaDatos.
    */
    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaDomicilioCliente.getSelectedRow();
        if(fila < 0){
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
        else{
            EliminaDato(idCliente);
            ConsultaDatos(idCliente);
        }
    }

    /*
        btnInsertarActionPerformed
        Evento de clic del botón de inserción.
        Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
        campos de entrada.
    */
    private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
        InsertaDato(idCliente);
        ConsultaDatos(idCliente);
    }

    /*
        btnModificarActionPerformed
        Evento de clic del botón de modificación.
        Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
        ConsultaDatos.
    */

```

```

private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaDomicilioCliente.getSelectedRow();
    if(fila < 0){
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
    else{
        ModificaDato(idCliente);
        ConsultaDatos(idCliente);
    }
}

}

public static void main(String args[]) {
    int idCliente = 1;
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new DomicilioCliente(idCliente).setVisible(true);
        }
    });
}
}

```

Proveedor

Id Proveedor	Nombre del Proveedor	Teléfono del Proveed...	Email del Proveedor
1	Alberto Hernandez	5674443422	alh@gmail.com

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Proveedor.

```
import java.sql.*;
```

```
import javax.swing.JOptionPane;
```

```
import javax.swing.table.DefaultTableModel;
```

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

```

// Librerías del sistema usadas para crear y gestionar el formulario Proveedor.
import java.sql.*;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/*
    Clase para gestionar el manejo de los proveedores dentro de los Servicios de
    Mantenimiento
    Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class Proveedor extends javax.swing.JFrame
{
    private String usuario;
    private String rol;

    /*
        Constructor del formulario Proveedor.
        Inicializa el formulario y carga los datos de la tabla Comercial.Proveedor en el
        JTable.
    */
    public Proveedor(String usuario, String rol)
    {
        this.usuario = usuario;
        this.rol = rol;
        initComponents();
        ConsultaDatos();
        configurarPermisos();
    }

    /*
        configurarPermisos
        Permite deshabilitar los botones de Insertar, Modificar y Eliminar para el
        usuario Empleado
    */
    public void configurarPermisos()
    {
        if(rol.equals("Empleado")){
            btnInsertar.setEnabled(false);
            btnModificar.setEnabled(false);
            btnEliminar.setEnabled(false);
        }
    }

    /*

```

ConsultaDatos

Recupera y muestra todos los registros de la tabla Comercial.Proveedor en el JTable.

Creación del modelo de la tabla agregando los encabezados de cada columna donde inicialmente no contiene filas.

```
*/
private void ConsultaDatos()
{
    String[] datos = new String[4];
    DefaultTableModel model = new DefaultTableModel(new String[]{
        "Id Proveedor", "Nombre del Proveedor", "Teléfono del Proveedor", "Email
del Proveedor"
    }, 0);
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        // Conexión a la base de datos de Postgres.
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        String sql = "SELECT * FROM Comercial.Proveedor";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();
        TablaProveedor.setModel(model);

        // Agrega los valores a una nueva fila para almacenarlos en el JTable.
        while (rs.next()) {
            datos[0] = String.valueOf(rs.getInt("idProveedor"));
            datos[1] = rs.getString("nombreProveedor");
            datos[2] = rs.getString("telefonoProveedor");
            datos[3] = rs.getString("emailProveedor");
            model.addRow(datos);
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al cargar los proveedores.");
    }
}

/*
InsertaDato
```

Inserta un nuevo registro en la tabla Comercial.Proveedor con los datos ingresados.

Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un mensaje de error.

```
*/
private void InsertaDato()
{
    String nombre = txtNombreProveedor.getText();
    String telefono = txtTelefonoProveedor.getText();
    String email = txtEmailProveedor.getText();

    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establece la conexión a la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Verifica si el RFC, teléfono o correo ya existen
        String verifica = "SELECT COUNT(*) FROM Comercial.Proveedor WHERE
telefonoProveedor = ? OR emailProveedor = ?";
        stmt = bd.prepareStatement(verifica);
        stmt.setString(1, telefono);
        stmt.setString(2, email);

        // Ejecuta la consulta
        rs = stmt.executeQuery();
        int count = 0;
        if (rs.next()) {
            count = rs.getInt(1);
        }

        if (count > 0) {
            JOptionPane.showMessageDialog(null, "El teléfono o el correo electrónico
ya están en uso.");
            txtNombreProveedor.setText("");
            txtTelefonoProveedor.setText("");
            txtEmailProveedor.setText("");
        } else {
            // Inserta el nuevo proveedor
```

```

        String sql = "INSERT INTO Comercial.Proveedor (nombreProveedor,
telefonoProveedor, emailProveedor) VALUES (?, ?, ?)";
        stmt.close();
        stmt = bd.prepareStatement(sql);
        stmt.setString(1, nombre);
        stmt.setString(2, telefono);
        stmt.setString(3, email);

        stmt.executeUpdate();

        txtNombreProveedor.setText("");
        txtTelefonoProveedor.setText("");
        txtEmailProveedor.setText("");

        rs.close();
        stmt.close();
        bd.close();
    }
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al insertar el Proveedor: ");
}
}

```

/*

ModificaDato

Modifica el registro seleccionado en la tabla Comercial.Proveedor con los datos actualizados.

Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un mensaje de error.

*/

```
private void ModificaDato(int idProveedor)
```

```
{
```

```
    String nombre = txtNombreProveedor.getText();
```

```
    String telefono = txtTelefonoProveedor.getText();
```

```
    String email = txtEmailProveedor.getText();
```

```
    Connection bd = null;
```

```
    PreparedStatement stmt = null;
```

```
    ResultSet rs = null;
```

```
    try {
```

```
        // Establece la conexión a la base de datos
```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Verifica si el RFC, teléfono o correo ya existen
        String verifica = "SELECT COUNT(*) FROM Comercial.Proveedor WHERE
telefonoProveedor = ? OR emailProveedor = ?";
        stmt = bd.prepareStatement(verifica);
        stmt.setString(1, telefono);
        stmt.setString(2, email);

        // Ejecuta la consulta
        rs = stmt.executeQuery();
        int count = 0;
        if (rs.next()) {
            count = rs.getInt(1);
        }

        if (count > 0) {
            JOptionPane.showMessageDialog(null, "El teléfono o el correo electrónico
ya están en uso.");
            txtNombreProveedor.setText("");
            txtTelefonoProveedor.setText("");
            txtEmailProveedor.setText("");
        } else {
            // Modifica el proveedor en la base de datos
            String sql = "UPDATE Comercial.Proveedor SET nombreProveedor = ?,
telefonoProveedor = ?, emailProveedor = ? WHERE idProveedor = ?";
            stmt = bd.prepareStatement(sql);
            stmt.setString(1, nombre);
            stmt.setString(2, telefono);
            stmt.setString(3, email);
            stmt.setInt(4, idProveedor);

            stmt.executeUpdate();

            txtNombreProveedor.setText("");
            txtTelefonoProveedor.setText("");
            txtEmailProveedor.setText("");

            stmt.close();
            bd.close();
        }
    } catch (SQLException ex) {

```

```

        JOptionPane.showMessageDialog(null, "Error al modificar el Proveedor.");
    }
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Comercial.Proveedor con los datos
actualizados.
*/
private void EliminaDato(int idProveedor)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        String sql = "DELETE FROM Comercial.Proveedor WHERE idProveedor =
?";

        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idProveedor);

        stmt.executeUpdate();

        txtNombreProveedor.setText("");
        txtTelefonoProveedor.setText("");
        txtEmailProveedor.setText("");

        stmt.close();
        bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al eliminar el Proveedor.");
    }
}

/*
SeleccionaDato
Selecciona los datos del proveedor y los muestra en los campos de texto
correspondientes.
*/
private void SeleccionaDato(int idProveedor)
{
    Connection bd = null;

```

```

        PreparedStatement stmt = null;
        ResultSet rs = null;
        try {
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
            String sql = "SELECT nombreProveedor, telefonoProveedor, emailProveedor
FROM Comercial.Proveedor WHERE idProveedor = ?";
            stmt = bd.prepareStatement(sql);
            stmt.setInt(1, idProveedor);
            rs = stmt.executeQuery();
            if (rs.next()) {
                txtNombreProveedor.setText(rs.getString("nombreProveedor"));
                txtTelefonoProveedor.setText(rs.getString("telefonoProveedor"));
                txtEmailProveedor.setText(rs.getString("emailProveedor"));
            }
            rs.close();
            stmt.close();
            bd.close();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null, "Error al obtener los datos del
Proveedor.");
            ex.printStackTrace();
        }
    }

    /*
        btnInsertarActionPerformed
        Evento de clic del botón de inserción.
        Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
    */
    private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
        InsertaDato();
        ConsultaDatos();
    }

    /*
        btnEliminarActionPerformed
        Evento de clic del botón de eliminación.
        Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a
ConsultaDatos.
    */
    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {

```

```

        int fila = TablaProveedor.getSelectedRow();
        if(fila >= 0){
            int idProveedor = Integer.parseInt(TablaProveedor.getValueAt(fila,
0).toString());
            EliminaDato(idProveedor);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
        btnModificarActionPerformed
        Evento de clic del botón de modificación.
        Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
ConsultaDatos.
    */
    private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaProveedor.getSelectedRow();
        if(fila >= 0){
            int idProveedor = Integer.parseInt(TablaProveedor.getValueAt(fila,
0).toString());
            ModificaDato(idProveedor);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
        TablaClienteMouseClicked
        Evento que se activa al hacer clic en una celda del JTable.
        Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
    */
    private void TablaProveedorMouseClicked(java.awt.event.MouseEvent evt) {
        int fila = TablaProveedor.getSelectedRow();
        if(fila >= 0) {
            int idProveedor = Integer.parseInt(TablaProveedor.getValueAt(fila,
0).toString());

```

```

        SeleccionaDato(idProveedor);
    }
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

public static void main(String args[]) {
    String usuario = "";
    String rol = "";
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Proveedor(usuario, rol).setVisible(true);
        }
    });
}
}

```

Material

idMaterial	Nombre del ...	Tipo del Mat...	Precio del M...	Existencia
1	Ladrillo	Electricidad	10	500

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Servicio.

```
import javax.swing.JOptionPane;
```

```
import javax.swing.table.TableModel;
```

```
public class Material extends javax.swing.JFrame
{
```

```
    private String usuario;
```

```
    private String rol;
```

```
    /* Constructor del formulario Material.
```

```
    Inicializa el formulario y carga los datos de la tabla Material en el JTable.*/
```

```

public Material(String usuario, String rol)
{
    this.usuario = usuario;
    this.rol = rol;
    initComponents();

    OMaterial objetoMaterial = new OMaterial();
    objetoMaterial.mostrarMaterial(TableMaterial);
    cBoxTipoMaterial.setSelectedIndex(-1);
    configurarPermisos();
}

/* configurarPermisos
    Permite deshabilitar los botones de Insertar, Modificar y Eliminar para el
usuario Empleado
*/
public void configurarPermisos()
{
    if(rol.equals("Empleado")){
        btnInsertar.setEnabled(false);
        btnModificar.setEnabled(false);
        btnEliminar.setEnabled(false);
    }
}

/*
    btnEliminarActionPerformed
    Evento de clic del botón de eliminación.
    Extrae el idMaterial, llama a EliminaDato, actualiza el JTable llamando a
ConsultaDatos.
*/
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    OMaterial objetoMaterial = new OMaterial();

    // Obtener el ID del renglón seleccionado
    int fila = TableMaterial.getSelectedRow();
    if (fila == -1) {
        JOptionPane.showMessageDialog(null, "Selecciona un renglón para
modificar.");
        return;
    }

    // Obtén el modelo de la tabla

```

```

TableModel modelo = TableMaterial.getModel();

// Obtén el idMaterial de la columna correspondiente (por ejemplo, columna 0)
int idMaterial;
try {
    Object valor = modelo.getValueAt(fila, 0);
    if (valor instanceof Integer) {
        idMaterial = (int) valor; // Si ya es un entero
    } else {
        idMaterial = Integer.parseInt(valor.toString()); // Si es texto, conviértelo a
entero
    }
} catch (Exception e) {
    JOptionPane.showMessageDialog(null, "Error al obtener el ID del material: "
+ e.getMessage());
    return;
}

// Llamar a modificarMaterial con los valores correctos
objetoMaterial.EliminarMaterial(idMaterial);

// Actualizar la tabla
objetoMaterial.mostrarMaterial(TableMaterial);
}

/*
TablaClienteMouseClicked
Evento que se activa al hacer clic en una celda del JTable.
Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
*/
private void TableMaterialMouseClicked(java.awt.event.MouseEvent evt) {
    OMaterial objetoMaterial = new OMaterial();
    objetoMaterial.SeleccionarMaterial(TableMaterial,txtNombreMaterial,
cBoxTipoMaterial, txtPrecioMaterial, txtExistencias);
}

/*
btnInsertarActionPerformed
Evento de clic del botón de inserción.
Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
*/

```

```

private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    OMaterial objetoMaterial = new OMaterial();
    objetoMaterial.insertarMaterial(txtNombreMaterial, cBoxTipoMaterial,
txtPrecioMaterial, txtExistencias);
    objetoMaterial.mostrarMaterial(TableMaterial);
}

/*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idServicio, llama a ModificaDato, actualiza el JTable llamando a
ConsultaDatos.
*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    OMaterial objetoMaterial = new OMaterial();

    // Obtener el ID del renglón seleccionado
    int fila = TableMaterial.getSelectedRow();
    if (fila == -1) {
        JOptionPane.showMessageDialog(null, "Selecciona un renglón para
modificar.");
        return;
    }

    // Obtén el modelo de la tabla
    TableModel modelo = TableMaterial.getModel();

    // Obtén el idMaterial de la columna correspondiente (por ejemplo, columna 0)
    int idMaterial;
    try {
        Object valor = modelo.getValueAt(fila, 0);
        if (valor instanceof Integer) {
            idMaterial = (int) valor; // Si ya es un entero
        } else {
            idMaterial = Integer.parseInt(valor.toString()); // Si es texto, conviértelo a
entero
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, "Error al obtener el ID del material: " +
e.getMessage());
        return;
    }

    // Llamar a modificarMaterial con los valores correctos

```

```
objetoMaterial.ModificarMaterial(txtNombreMaterial, cBoxTipoMaterial,
txtPrecioMaterial, txtExistencias, idMaterial);
```

```
// Actualizar la tabla
objetoMaterial.mostrarMaterial(TableMaterial);
}
```

```
public static void main(String args[]) {
    String usuario = "";
    String rol = "";
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Material(usuario, rol).setVisible(true);
        }
    });
}
```

Trabajador

Id Trabajador	RFC del Tra...	Nombre del ...	Teléfono del ...	Email del Tr...

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Servicio.

```
import java.util.List;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
```

```

/*
    Clase para gestionar el manejo de los trabajadores dentro de los Servicios de
    Mantenimiento
    Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class Trabajador extends javax.swing.JFrame
{
    private String usuario;
    private String rol;
    int idTrabajador = -1;

    // Lista que almacena objetos de tipo DetalleTrabajador.
    private List<DetalleTrabajador> detalleTrabajadorForms = new
    ArrayList<DetalleTrabajador>();

    /*
        Constructor del formulario DetalleTrabajador.
        Inicializa el formulario y carga los datos de la tabla DetalleTrabajador en el
        JTable.
    */
    public Trabajador(String usuario, String rol)
    {
        this.usuario = usuario;
        this.rol = rol;
        initComponents();
        ConsultaDatos();
        configurarPermisos();
    }

    /*
        configurarPermisos
        Permite deshabilitar los botones de Insertar, Modificar y Eliminar para el
        usuario Empleado
    */
    public void configurarPermisos()
    {
        if(rol.equals("Empleado")){
            btnInsertar.setEnabled(false);
            btnModificar.setEnabled(false);
            btnEliminar.setEnabled(false);
        }
    }
}

```

```

/*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Operativo.Trabajador en el
    JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
*/
private void ConsultaDatos()
{
    String[] datos = new String[5];
    DefaultTableModel model = new DefaultTableModel(new String[]{
        "Id Trabajador", "RFC del Trabajador", "Nombre del Trabajador", "Teléfono
del Trabajador", "Email del Trabajador"
    }, 0);
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        // Conexión a la base de datos de Postgres.
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimi
miento", "postgres", "postgres");
        String sql = "SELECT * FROM Operativo.Trabajador";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();
        TablaTrabajador.setModel(model);

        // Agrega los valores a una nueva fila para almacenarlos en el JTable.
        while (rs.next()) {
            datos[0] = String.valueOf(rs.getInt("idTrabajador"));
            datos[1] = rs.getString("RFCTrabajador");
            datos[2] = rs.getString("nombreTrabajador");
            datos[3] = rs.getString("telefonoTrabajador");
            datos[4] = rs.getString("emailTrabajador");
            model.addRow(datos);
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al cargar los trabajadores.");
    }
}

```

```

/*
    InsertaDato
    Inserta un nuevo registro en la tabla Operativo.Trabajador con los datos
    ingresados.
    Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un
    mensaje de error.
*/
private void InsertaDato()
{
    String rfc = txtRFCTrabajador.getText();
    String nombre = txtNomTrabajador.getText();
    String telefono = txtTelTrabajador.getText();
    String email = txtEmailTrabajador.getText();

    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establece la conexión a la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Verifica si el RFC, teléfono o correo ya existen
        String verifica = "SELECT COUNT(*) FROM Operativo.Trabajador WHERE
RFCTrabajador = ? OR telefonoTrabajador = ? OR emailTrabajador= ?";
        stmt = bd.prepareStatement(verifica);
        stmt.setString(1, rfc);
        stmt.setString(2, telefono);
        stmt.setString(3, email);

        // Ejecuta la consulta
        rs = stmt.executeQuery();
        int count = 0;
        if (rs.next()) {
            count = rs.getInt(1);
        }

        if (count > 0) {
            JOptionPane.showMessageDialog(null, "El RFC, teléfono o correo
electrónico ya están en uso.");
            txtRFCTrabajador.setText("");
            txtNomTrabajador.setText("");

```

```

        txtTelTrabajador.setText("");
        txtEmailTrabajador.setText("");
    } else {
        // Inserta el nuevo proveedor
        String sql = "INSERT INTO Operativo.Trabajador (RFCTrabajador,
nombreTrabajador, telefonoTrabajador, emailTrabajador) VALUES (?, ?, ?,?)";
        stmt.close();
        stmt = bd.prepareStatement(sql);
        stmt.setString(1, rfc);
        stmt.setString(2, nombre);
        stmt.setString(3, telefono);
        stmt.setString(4, email);

        stmt.executeUpdate();
        txtRFCTrabajador.setText("");
        txtNomTrabajador.setText("");
        txtTelTrabajador.setText("");
        txtEmailTrabajador.setText("");

        rs.close();
        stmt.close();
        bd.close();
    }
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al insertar el trabajador: " +
ex.getMessage());
}
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Operativo.Trabajador con los datos
actualizados.
Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un
mensaje de error.
*/
private void ModificaDato(int idTrabajador)
{
    String rfc = txtRFCTrabajador.getText();
    String nombre = txtNomTrabajador.getText();
    String telefono = txtTelTrabajador.getText();
    String email = txtEmailTrabajador.getText();

    Connection bd = null;

```

```

PreparedStatement stmt = null;
ResultSet rs = null;

try {
    // Establece la conexión a la base de datos
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

    // Verifica si el RFC, teléfono o correo ya existen
    String verifica = "SELECT COUNT(*) FROM Operativo.Trabajador WHERE
RFCTrabajador = ? OR telefonoTrabajador = ? OR emailTrabajador= ?";
    stmt = bd.prepareStatement(verifica);
    stmt.setString(1, rfc);
    stmt.setString(2, telefono);
    stmt.setString(3, email);

    // Ejecuta la consulta
    rs = stmt.executeQuery();
    int count = 0;
    if (rs.next()) {
        count = rs.getInt(1);
    }

    if (count > 0 && !nombre.equals(txtNomTrabajador.getText())) {
        JOptionPane.showMessageDialog(null, "El RFC, teléfono o correo electrónico ya están en uso.");
        txtRFCTrabajador.setText("");
        txtNomTrabajador.setText("");
        txtTelTrabajador.setText("");
        txtEmailTrabajador.setText("");
    } else {
        // Modifica el proveedor en la base de datos
        String sql = "UPDATE Operativo.Trabajador SET RFCTrabajador =
?,nombreTrabajador = ?, telefonoTrabajador = ?, emailTrabajador = ? WHERE
idTrabajador = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setString(1, rfc);
        stmt.setString(2, nombre);
        stmt.setString(3, telefono);
        stmt.setString(4, email);
        stmt.setInt(5,idTrabajador);

        stmt.executeUpdate();
    }
} catch (SQLException e) {
    e.printStackTrace();
}

```

```

        txtRFCTrabajador.setText("");
        txtNomTrabajador.setText("");
        txtTelTrabajador.setText("");
        txtEmailTrabajador.setText("");

        stmt.close();
        bd.close();
    }
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al modificar el trabajador.");
}
}

/*
    EliminaDato
    Elimina el registro seleccionado en la tabla Operativo.Trabajador con los datos
    actualizados.
*/
private void EliminaDato(int idTrabajador)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        String sql = "DELETE FROM Operativo.Trabajador WHERE idTrabajador =
?";

        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idTrabajador);

        stmt.executeUpdate();

        txtRFCTrabajador.setText("");
        txtNomTrabajador.setText("");
        txtTelTrabajador.setText("");
        txtEmailTrabajador.setText("");

        stmt.close();
        bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al eliminar el trabajador.");
    }
}

```

```

    }
}

/*
    SeleccionaDato
    Selecciona los datos del trabajador y los muestra en los campos de texto
    correspondientes.
*/
private void SeleccionaDato(int idTrabajador)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        String sql = "SELECT RFCTrabajador, nombreTrabajador,
telefonoTrabajador, emailTrabajador FROM Operativo.Trabajador WHERE
idTrabajador = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idTrabajador);
        rs = stmt.executeQuery();
        if (rs.next()) {
            txtRFCTrabajador.setText(rs.getString("RFCTrabajador"));
            txtNomTrabajador.setText(rs.getString("nombreTrabajador"));
            txtTelTrabajador.setText(rs.getString("telefonoTrabajador"));
            txtEmailTrabajador.setText(rs.getString("emailTrabajador"));
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al obtener los datos del
trabajador.");
        ex.printStackTrace();
    }
}

/*
    btnDetalleTrabajadorActionPerformed
    Evento que se dispara al hacer clic en el botón de Detalle del Trabajador.

```

```

        Abre el formulario DetalleTrabajador para gestionar los detalles trabajadores.
    */
    private void btnDetalleTrabajadorActionPerformed(java.awt.event.ActionEvent evt)
    {
        if(idTrabajador != -1)
        {
            DetalleTrabajador detalleTrabajadorForm = new
DetalleTrabajador(idTrabajador);
            detalleTrabajadorForms.add(detalleTrabajadorForm);
            detalleTrabajadorForm.setVisible(true);
        }
    }

    /*
        TablaClienteMouseClicked
        Evento que se activa al hacer clic en una celda del JTable.
        Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
    */
    private void TablaTrabajadorMouseClicked(java.awt.event.MouseEvent evt) {
        int fila = TablaTrabajador.getSelectedRow();
        if(fila >= 0) {
            idTrabajador = Integer.parseInt(TablaTrabajador.getValueAt(fila,
0).toString());
            SeleccionaDato(idTrabajador);

            if(!rol.equals("Empleado")){
                btnDetalleTrabajador.setEnabled(true);
            }
        }
        else {
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
        btnInsertarActionPerformed
        Evento de clic del botón de inserción.
        Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
    */
    private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
        InsertaDato();
    }

```

```

        ConsultaDatos();
    }

    /*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idTrabajador, llama a ModificaDato, actualiza el JTable llamando a
    ConsultaDatos.
    */
    private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaTrabajador.getSelectedRow();
        if(fila >= 0){
            int idTrabajador = Integer.parseInt(TablaTrabajador.getValueAt(fila,
0).toString());
            ModificaDato(idTrabajador);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    /*
    btnEliminarActionPerformed
    Evento de clic del botón de eliminación.
    Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a
    ConsultaDatos.
    */
    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaTrabajador.getSelectedRow();
        if(fila >= 0){
            int idTrabajador = Integer.parseInt(TablaTrabajador.getValueAt(fila,
0).toString());
            EliminaDato(idTrabajador);
            ConsultaDatos();
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
        }
    }

    public static void main(String args[]) {

```

```

String usuario = "";
String rol = "";
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new Trabajador(usuario, rol).setVisible(true);
    }
});
}

```

DetalleTrabajador

Trabajador	Profesión	Costo Base
Juan Pérez	Carpintero	450.00
Juan Pérez	Electricista	380.00

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Servicio.

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

```

```
/*
```

Clase para gestionar el manejo de los detalles de los trabajadores dentro de los Servicios de Mantenimiento

Proporciona métodos para consultar, insertar, modificar y eliminar datos.

```
*/
```

```

public class DetalleTrabajador extends javax.swing.JFrame
{

```

```
    int idTrabajador = -1;
```

```
/*
```

Constructor del formulario DetalleTrabajador.

Inicializa el formulario y carga los datos de la tabla DetalleTrabajador en el JTable.

```
*/  
public DetalleTrabajador(int idTrabajador)  
{  
    initComponents();  
    this.idTrabajador = idTrabajador;  
    ConsultaDatos(idTrabajador);  
}
```

/*
ConsultaDatos
Recupera y muestra todos los registros de la tabla Operativo.DetalleTrabajador en el JTable.

Creación del modelo de la tabla agregando los encabezados de cada columna donde inicialmente no contiene filas.

```
*/  
private void ConsultaDatos(int idTrabajador)  
{  
    String[] datos = new String[3];  
    DefaultTableModel model = new DefaultTableModel(new String[]{"Trabajador",  
"Profesión", "Costo Base"}, 0);  
    Connection bd = null;  
    PreparedStatement stmt = null;  
    ResultSet rs = null;  
  
    try {  
        bd =  
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");  
  
        // Consulta con JOIN para obtener el nombre del trabajador  
        String sql = "SELECT dt.idTrabajador, dt.profesion, dt.costoBase,  
t.nombreTrabajador "  
            + "FROM Operativo.DetalleTrabajador dt "  
            + "JOIN Operativo.Trabajador t ON dt.idTrabajador = t.idTrabajador "  
            + "WHERE dt.idTrabajador = ?";  
        stmt = bd.prepareStatement(sql);  
        stmt.setInt(1, idTrabajador);  
        rs = stmt.executeQuery();  
  
        TablaDetalleTrabajador.setModel(model);  
  
        while (rs.next())
```

```

{
    String nombreTrabajador = rs.getString("nombreTrabajador");
    String trabajador = nombreTrabajador;
    datos[0] = trabajador;
    datos[1] = rs.getString("profesion");
    datos[2] = rs.getString("costoBase") != null ? rs.getString("costoBase") :
"0.00";
    model.addRow(datos);
}
// Cerrar conexiones.
rs.close();
stmt.close();
bd.close();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar el Material del
Servicio: " + ex.getMessage());
}
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Operativo.DetalleTrabajador con los datos
ingresados.
Si ocurre un error(como duplicación de la profesión), muestra un mensaje de
error.
*/
private void InsertaDato(int idTrabajador)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establecer conexión con la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimi
miento", "postgres", "postgres");

        // Obtener datos del formulario
        String profesion = txtProfesion.getText();
        String costoB = txtCostoBase.getText();
        float costoBase = Float.parseFloat(costoB);

        //Verifica si profesión ya existe.

```

```

        String verificaProfesion = "SELECT COUNT(*) FROM
Operativo.DetalleTrabajador WHERE idTrabajador = ? AND profesion = ?";
        stmt = bd.prepareStatement(verificaProfesion);
        stmt.setInt(1, idTrabajador);
        stmt.setString(2, profesion);
        rs = stmt.executeQuery();

        if (rs.next() && rs.getInt(1) > 0) {
            JOptionPane.showMessageDialog(null, "Este trabajador ya tiene asignada
esta profesión.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String sql = "INSERT INTO Operativo.DetalleTrabajador (idTrabajador,
profesion, costoBase) VALUES (?, ?, ?)";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idTrabajador); // Usa el idTrabajador almacenado en la clase
        stmt.setString(2, profesion);
        stmt.setFloat(3, costoBase);

        stmt.executeUpdate();

        // Limpia los campos después de insertar
        txtProfesion.setText("");
        txtCostoBase.setText("");

    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, "Error al insertar el detalle del
trabajador: " + ex.getMessage());
    } finally {
        try {
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Operativo.DetalleTrabajador con
los datos actualizados.

```

Si ocurre un error(como duplicación de la profesión), muestra un mensaje de error.

```
*/
private void ModificaDato(int idTrabajador)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    String profesionActual = "";

    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        int fila = TablaDetalleTrabajador.getSelectedRow();
        if(fila >= 0) {
            profesionActual = TablaDetalleTrabajador.getValueAt(fila, 1).toString();
        }

        // Obtener datos del formulario
        String profesion = txtProfesion.getText();
        String costoB = txtCostoBase.getText();
        float costoBase = Float.parseFloat(costoB);

        //Verifica si profesión ya existe.
        String verificaProfesion = "SELECT COUNT(*) FROM
Operativo.DetalleTrabajador WHERE idTrabajador = ? AND profesion = ?";
        stmt = bd.prepareStatement(verificaProfesion);
        stmt.setInt(1, idTrabajador);
        stmt.setString(2, profesion);
        rs = stmt.executeQuery();

        if (rs.next() && rs.getInt(1) > 0) {
            JOptionPane.showMessageDialog(null, "Este trabajador ya tiene asignada
esta profesión.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String sql = "UPDATE Operativo.DetalleTrabajador SET profesion = ?,
costoBase = ? WHERE idTrabajador = ? AND profesion = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setString(1, profesion);
        stmt.setFloat(2, costoBase);
    }
}
```

```

        stmt.setInt(3, idTrabajador);
        stmt.setString(4, profesionActual);
        stmt.executeUpdate();

        txtProfesion.setText("");
        txtCostoBase.setText("");

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al modificar el detalle del
trabajador: " + e.getMessage());
    } finally {
        try {
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
    EliminaDato
    Elimina el registro seleccionado en la tabla Operativo.DetalleTrabajador con los
datos actualizados.
*/
private void EliminaDato(int idTrabajador)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        String profesion = txtProfesion.getText();

        String sql = "DELETE FROM Operativo.DetalleTrabajador WHERE
idTrabajador = ? AND profesion = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idTrabajador);
        stmt.setString(2, profesion);
        stmt.executeUpdate();

```

```

        txtProfesion.setText("");
        txtCostoBase.setText("");
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al eliminar el trabajador: " +
ex.getMessage());
    } finally {
        try {
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null, "Error al eliminar el Servicio.");
        }
    }
}
}

```

/*
 btnInsertarActionPerformed
 Evento de clic del botón de inserción.
 Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los campos de entrada.

```

*/
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    InsertaDato(idTrabajador);
    ConsultaDatos(idTrabajador);
}

```

/*
 btnModificarActionPerformed
 Evento de clic del botón de modificación.
 Extrae el idTrabajador, llama a ModificaDato, actualiza el JTable llamando a ConsultaDatos.

```

*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaDetalleTrabajador.getSelectedRow();
    if(fila >= 0){
        idTrabajador = Integer.parseInt(TablaDetalleTrabajador.getValueAt(fila,
0).toString());
        ModificaDato(idTrabajador);
        ConsultaDatos(idTrabajador);
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila válida.");
    }
}

```

```

    }
}

/*
    btnEliminarActionPerformed
    Evento de clic del botón de eliminación.
    Extrae el idTrabajador, llama a EliminaDato, actualiza el JTable llamando a
    ConsultaDatos.
*/
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaDetalleTrabajador.getSelectedRow();
    if(fila >= 0){
        idTrabajador = Integer.parseInt(TablaDetalleTrabajador.getValueAt(fila,
0).toString());
        EliminaDato(idTrabajador);
        ConsultaDatos(idTrabajador);
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    TablaClienteMouseClicked
    Evento que se activa al hacer clic en una celda del JTable.
*/
private void TablaDetalleTrabajadorMouseClicked(java.awt.event.MouseEvent
evt) {
    int fila = TablaDetalleTrabajador.getSelectedRow();
    if (fila >= 0) {
        String profesion = TablaDetalleTrabajador.getValueAt(fila, 1).toString();
        txtProfesion.setText(profesion);

        String costoBase = TablaDetalleTrabajador.getValueAt(fila, 2).toString();
        txtCostoBase.setText(costoBase);
    } else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila.");
    }
}
}

```

Servicio

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

package Tablas;

// Librerías del sistema usadas para crear y gestionar el formulario Servicio.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.util.Date;
import javax.swing.JComboBox;
import com.toedter.calendar.JDateChooser;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.List;
```

/*

Clase para gestionar el manejo de los servicio dentro de los Servicios de Mantenimiento

Proporciona métodos para consultar, insertar, modificar y eliminar datos.

*/

```
public class Servicio extends javax.swing.JFrame
```

```
{
```

```
    private String usuario;
```

```
    private String rol;
```

```
    int idServicio = -1;
```

```
    // Lista que almacena objetos de tipo TrabajadorXServicio.
```

```
    private List<TrabajadorXServicio> trabajadorXServicioForms = new
    ArrayList<TrabajadorXServicio>();
```

```

// Lista que almacena objetos de tipo MaterialXServicio.
private List<MaterialXServicio> materialXServicioForms = new
ArrayList<MaterialXServicio>();

/*
    Constructor del formulario Servicio.
    Inicializa el formulario y carga los datos de la tabla Servicio en el JTable.
*/
public Servicio(String usuario, String rol)
{
    this.usuario = usuario;
    this.rol = rol;
    initComponents();
    ConsultaDatos();
    cargarDomicilios();
    cmbxDomicilio.setSelectedIndex(-1);
    configurarPermisos();
}

/*
    configurarPermisos
    Permite deshabilitar los botones de Insertar, Modificar y Eliminar para el
usuario Empleado
*/
public void configurarPermisos()
{
    if(rol.equals("Empleado")){
        btnInsertar.setEnabled(false);
        btnModificar.setEnabled(false);
        btnEliminar.setEnabled(false);
    }
}

/*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Comercial.Compra en el
JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
donde inicialmente no contiene filas.
*/
private void ConsultaDatos()
{
    String[] datos = new String[7];

```

```

DefaultTableModel model = new DefaultTableModel(new String[]{"Id Servicio",
"Domicilio", "Fecha de Cotización", "Fecha de Inicio", "Fecha de Finalización",
"Duración", "Costo Final"}, 0);
Connection bd = null;
PreparedStatement stmt = null;
ResultSet rs = null;
try
{
    // Conexión a la base de datos de Postgres.
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
    String sql = "SELECT s.idServicio, s.idDomicilio, s.fechaCotizacion,
s.fechaInicio, s.fechaFinal, s.duracion, s.costoFinal, "
+ "CONCAT(dc.domicilioCliente, ' (' , c.nombreCliente, ')') AS
DomicilioCliente "
+ "FROM Operativo.Servicio s JOIN Comercial.DomicilioCliente dc ON
s.idDomicilio = dc.idDomicilio "
+ "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente";
    stmt = bd.prepareStatement(sql);
    rs = stmt.executeQuery();
    TablaServicio.setModel(model);

    // Agrega los valores a una nueva fila para almacenarlos en el JTable.
    while(rs.next()){
        datos[0] = String.valueOf(rs.getInt("idServicio"));
        datos[1] = rs.getString("DomicilioCliente");
        datos[2] = rs.getString("fechaCotizacion");
        datos[3] = rs.getString("fechaInicio");
        datos[4] = rs.getString("fechaFinal");
        datos[5] = rs.getString("duracion");
        datos[6] = rs.getString("costoFinal") != null ? rs.getString("costoFinal") : "0.00";
        model.addRow(datos);
    }
    rs.close();
    stmt.close();
    bd.close();
} catch(SQLException ex){
    JOptionPane.showMessageDialog(null, "Error al cargar el Cliente.");
}
}

/*

```

Carge la información de la tabla Comercial.Cliente y Comercial.DomicilioCliente y la muestra en un combobox.

```
*/
public void cargarDomicilios()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos de Postgres.
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
        "postgres");
        String sql = "SELECT dc.domicilioCliente, c.nombreCliente " +
            "FROM Comercial.DomicilioCliente dc " +
            "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();

        // Limpiar el JComboBox antes de agregar datos
        cmbboxDomicilio.removeAllItems();

        // Leer cada domicilio y cliente del resultado de la consulta
        while (rs.next()) {
            String domicilioCliente = rs.getString("domicilioCliente");
            String nombreCliente = rs.getString("nombreCliente");
            String domicilio = domicilioCliente + " (" + nombreCliente + ")";

            // Agregar resultado al JComboBox
            cmbboxDomicilio.addItem(domicilio);
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cargar los domicilios.");
    }
}

/*
InsertaDato
```

Inserta un nuevo registro en la tabla Operativo.Servicio con los datos ingresados.

```
*/
public void InsertaDato()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Obtener datos del formulario
        String domicilioSeleccionado = cmbboxDomicilio.getSelectedItem().toString();
        String[] partesDomicilio = domicilioSeleccionado.split("\\(");
        String domicilioCliente = partesDomicilio[0].trim();

        // Validar si las fechas de inicio y finalización están seleccionadas
        if (dtpFech_Ini.getDate() == null || dtpFech_Fin.getDate() == null) {
            JOptionPane.showMessageDialog(null, "Por favor, seleccione todas las fechas.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        java.sql.Date fechaCotizacion = new java.sql.Date(new Date().getTime());
        java.sql.Date fechaInicio = new java.sql.Date(dtpFech_Ini.getDate().getTime());
        java.sql.Date fechaFinal = new java.sql.Date(dtpFech_Fin.getDate().getTime());

        // Validar fechas de inicio y finalización
        Date fechaActual = new Date();
        if (dtpFech_Ini.getDate().before(fechaActual)) {
            JOptionPane.showMessageDialog(null, "La fecha de inicio no puede ser anterior a la fecha actual (cotización).", "Error de Fecha", JOptionPane.ERROR_MESSAGE);
            return;
        }

        if (dtpFech_Fin.getDate().before(dtpFech_Ini.getDate())) {
            JOptionPane.showMessageDialog(null, "La fecha de finalización no puede ser anterior a la fecha de inicio.", "Error de Fecha", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Conexión a la base de datos de Postgres.
```

```
bd =  
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
```

```
long idDomicilio = 0;  
String queryIdDomicilio = "SELECT idDomicilio FROM Comercial.DomicilioCliente WHERE domicilioCliente = ?";  
stmt = bd.prepareStatement(queryIdDomicilio);  
stmt.setString(1, domicilioCliente);  
rs = stmt.executeQuery();
```

```
if (rs.next()) {  
    idDomicilio = rs.getLong("idDomicilio");  
} else {  
    JOptionPane.showMessageDialog(null, "El domicilio seleccionado no es válido.", "Error", JOptionPane.ERROR_MESSAGE);  
    return;  
}
```

```
// Verificar si ya existe un servicio programado para este domicilio  
String queryCheckServicio = "SELECT COUNT(*) FROM Operativo.Servicio WHERE idDomicilio = ? AND fechaFinal >= ?";  
stmt = bd.prepareStatement(queryCheckServicio);  
stmt.setLong(1, idDomicilio);  
stmt.setDate(2, fechaInicio);  
rs = stmt.executeQuery();
```

```
if (rs.next() && rs.getInt(1) > 0) {  
    JOptionPane.showMessageDialog(null, "No se puede programar un nuevo servicio para este domicilio hasta que el servicio actual finalice.", "Error", JOptionPane.ERROR_MESSAGE);  
    return;  
}
```

```
String insertInfo = "INSERT INTO Operativo.Servicio(idDomicilio, fechaCotizacion, fechaInicio, fechaFinal) VALUES (?, ?, ?, ?)";  
stmt = bd.prepareStatement(insertInfo);  
stmt.setLong(1, idDomicilio);  
stmt.setDate(2, fechaCotizacion);  
stmt.setDate(3, fechaInicio);  
stmt.setDate(4, fechaFinal);  
stmt.executeUpdate();
```

```
comboBoxDomicilio.setSelectedIndex(-1);
```

```

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al insertar el servicio: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
    ModificaDato
    Modifica el registro seleccionado en la tabla Operativo.Servicio con los datos
actualizados.
*/
private void ModificaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Obtener datos del formulario
        String domicilioSeleccionado = (String) cmbxDomicilio.getSelectedItem();
        if (domicilioSeleccionado == null || domicilioSeleccionado.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Seleccione un domicilio válido.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String[] partesDomicilio = domicilioSeleccionado.split("\\(");

        // Validar fechas
        if (dtpFech_Ini.getDate() == null || dtpFech_Fin.getDate() == null) {
            JOptionPane.showMessageDialog(null, "Por favor, seleccione todas las
fechas.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }
}

```

```

        java.sql.Date fechaInicio = new
java.sql.Date(dtpFech_Ini.getDate().getTime());
        java.sql.Date fechaFinal = new
java.sql.Date(dtpFech_Fin.getDate().getTime());

        if (fechaFinal.before(fechaInicio)) {
            JOptionPane.showMessageDialog(null, "La fecha de finalización no puede
ser anterior a la fecha de inicio.", "Error de Fecha",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Conexión a la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener idDomicilio
        long idDomicilio = 0;
        String queryIdDomicilio = "SELECT dc.idDomicilio FROM
Comercial.DomicilioCliente dc JOIN Comercial.Cliente c ON dc.idCliente =
c.idCliente "
            + "WHERE dc.domicilioCliente = ? AND c.nombreCliente = ?";
        stmt = bd.prepareStatement(queryIdDomicilio);
        String domicilioCliente = partesDomicilio[0].trim();
        String nombreCliente = partesDomicilio[1].replace(" ", "").trim();

        stmt.setString(1, domicilioCliente);
        stmt.setString(2, nombreCliente);
        rs = stmt.executeQuery();

        if (rs.next()) {
            idDomicilio = rs.getLong("idDomicilio");
        } else {
            JOptionPane.showMessageDialog(null, "El domicilio seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Verificar si ya existe un servicio programado en el mismo rango de fechas
y domicilio
        String queryCheckServicio = "SELECT COUNT(*) FROM Operativo.Servicio
WHERE idDomicilio = ? AND fechaFinal >= ? AND fechaInicio <= ? "

```

```

        + "AND idServicio <> ?";
stmt = bd.prepareStatement(queryCheckServicio);
stmt.setLong(1, idDomicilio);
stmt.setDate(2, fechaInicio);
stmt.setDate(3, fechaFinal);
stmt.setInt(4, idServicio);

rs = stmt.executeQuery();

if (rs.next() && rs.getInt(1) > 0) {
    JOptionPane.showMessageDialog(null,
        "Ya existe otro servicio programado en este rango de fechas para el
mismo domicilio.",
        "Error",
        JOptionPane.ERROR_MESSAGE);
    return;
}

// Insertar nuevo servicio
String insertInfo = "UPDATE Operativo.Servicio SET idDomicilio = ?,
fechaInicio = ?, fechaFinal= ? WHERE idServicio = ?";
stmt = bd.prepareStatement(insertInfo);
stmt.setLong(1, idDomicilio);
stmt.setDate(2, fechaInicio);
stmt.setDate(3, fechaFinal);
stmt.setInt(4, idServicio);
stmt.executeUpdate();

cmbxDomicilio.setSelectedIndex(-1);
} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al insertar el servicio: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}
}

```

```

/*
    EliminaDato
    Elimina el registro seleccionado en la tabla Operativo.Servicio con los datos
actualizados.
*/
private void EliminaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try
    {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
        String sql = "DELETE FROM Operativo.Servicio WHERE idServicio = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idServicio);

        stmt.executeUpdate();

        cmbxDomicilio.setSelectedIndex(-1);
        stmt.close();
        bd.close();
    }catch(SQLException ex){
        JOptionPane.showMessageDialog(null, "Error al eliminar el Servicio.");
    }
}

/*
    SeleccionaDato
    Selecciona los datos del servicio y los muestra en los campos de texto
correspondientes.
*/
private void SeleccionaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos

```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Consulta para obtener los datos del servicio seleccionado
        String query = "SELECT s.idServicio, s.idDomicilio, s.fechaCotizacion, s.fechaInicio, s.fechaFinal, s.duracion, s.costoFinal, "
            + "CONCAT(dc.domicilioCliente, ' (', c.nombreCliente, ')') AS DomicilioCliente "
            + "FROM Operativo.Servicio s "
            + "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio "
            + "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente "
            + "WHERE s.idServicio = ?";
        stmt = bd.prepareStatement(query);
        stmt.setInt(1, idServicio);
        rs = stmt.executeQuery();

        // Si encontramos un servicio con ese id
        if (rs.next()) {
            // Obtener los valores de la consulta
            String domicilioCliente = rs.getString("DomicilioCliente");
            java.sql.Date fechaInicio = rs.getDate("fechaInicio");
            java.sql.Date fechaFinal = rs.getDate("fechaFinal");

            cmbboxDomicilio.setSelectedItem(domicilioCliente);
            dtpFech_Ini.setDate(fechaInicio);
            dtpFech_Fin.setDate(fechaFinal);
        }

        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al seleccionar el servicio: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

    }
}

/*
    btnInsertarActionPerformed
    Evento de clic del botón de inserción.
    Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
*/
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    InsertaDato();
    ConsultaDatos();
}

/*
    btnEliminarActionPerformed
    Evento de clic del botón de eliminación.
    Extrae el idServicio, llama a EliminaDato, actualiza el JTable llamando a
ConsultaDatos.
*/
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaServicio.getSelectedRow();
    if(fila >= 0){
        int idServicio = Integer.parseInt(TablaServicio.getValueAt(fila, 0).toString());
        EliminaDato(idServicio);
        ConsultaDatos();
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idServicio, llama a ModificaDato, actualiza el JTable llamando a
ConsultaDatos.
*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaServicio.getSelectedRow();
    if(fila >= 0){
        int idServicio = Integer.parseInt(TablaServicio.getValueAt(fila, 0).toString());

```

```

        ModificaDato(idServicio);
        ConsultaDatos();
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    TablaClienteMouseClicked
    Evento que se activa al hacer clic en una celda del JTable.
    Llama al método SeleccionaDato para cargar los datos del servicio en los
campos de texto.
*/
private void TablaServicioMouseClicked(java.awt.event.MouseEvent evt) {
    int fila = TablaServicio.getSelectedRow();
    if(fila >= 0) {
        idServicio = Integer.parseInt(TablaServicio.getValueAt(fila, 0).toString());
        SeleccionaDato(idServicio);

        if(!rol.equals("Empleado")){
            btnTrabajador.setEnabled(true);
            btnMaterial.setEnabled(true);
        }
    }
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    btnTrabajadorActionPerformed
    Evento que se dispara al hacer clic en el botón de Trabajador del Servicio.
    Abre el formulario TrabajadorXServicio para gestionar los trabajadores
asociados a un servicio específica.
*/
private void btnTrabajadorActionPerformed(java.awt.event.ActionEvent evt) {
    if(idServicio != -1)
    {
        TrabajadorXServicio trabajadorXServicioForm = new
TrabajadorXServicio(idServicio);
        trabajadorXServicioForms.add(trabajadorXServicioForm);
    }
}

```

```

        trabajadorXServicioForm.addWindowListener(new
java.awt.event.WindowAdapter() {
    public void windowClosing(java.awt.event.WindowEvent e) {
        trabajadorXServicioForms.remove(trabajadorXServicioForm);
    }
});
    trabajadorXServicioForm.setVisible(true);
}
}

/*
    btnMaterial1ActionPerformed
    Evento que se dispara al hacer clic en el botón de Material del Servicio.
    Abre el formulario MaterialXServicio para gestionar los materiales asociados a
    un servicio específica.
*/
private void btnMaterialActionPerformed(java.awt.event.ActionEvent evt) {
    if(idServicio != -1)
    {
        MaterialXServicio materialXServicioForm = new
MaterialXServicio(idServicio);
        materialXServicioForms.add(materialXServicioForm);
        materialXServicioForm.addWindowListener(new
java.awt.event.WindowAdapter() {
            public void windowClosing(java.awt.event.WindowEvent e) {
                materialXServicioForms.remove(materialXServicioForm);
            }
        });
        materialXServicioForm.setVisible(true);
    }
}

public static void main(String args[]) {
    String usuario = "";
    String rol = "";
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Servicio(usuario, rol).setVisible(true);
        }
    });
}
}

```

MaterialXServicio

Servicio	Material	Cantidad	Subtotal Material
1 - Jenni Ruby ...	Ladrillo - Electr...	20	200.00

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

package Tablas;

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
```

/*

Clase para gestionar el manejo de los materiales de los servicios dentro de los Servicios de Mantenimiento

Proporciona métodos para consultar, insertar, modificar y eliminar datos.

*/

```
public class MaterialXServicio extends javax.swing.JFrame
{
```

```
    int idServicio = -1;
```

/*

Constructor del formulario MaterialXServicio.

Inicializa el formulario y carga los datos de la tabla MaterialXServicio en el JTable.

*/

```
public MaterialXServicio(int idServicio)
{
    initComponents();
    this.idServicio = idServicio;
    ConsultarDatos(idServicio);
    CargarMateriales();
    cmbMaterial.setSelectedIndex(-1);
```

```

    }

    /*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Operativo.MaterialXServicio
    en el JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
    */
    private void ConsultaDatos(int idServicio)
    {
        String[] datos = new String[4];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Servicio",
"Material", "Cantidad", "Subtotal Material"}, 0);
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            // Conexión a la base de datos.
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

            // Consulta SQL con JOINS.
            String sql = "SELECT s.idServicio, CONCAT(s.idServicio, ' - ', c.nombreCliente)
AS ServicioCliente, "
                + "mat.NombreMaterial, mat.TipoMaterial, m.cantMaterial,
m.subtotalMaterial "
                + "FROM Operativo.MaterialXServicio m "
                + "JOIN Operativo.Servicio s ON m.idServicio = s.idServicio "
                + "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio "
                + "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente "
                + "JOIN Operativo.Material mat ON m.idMaterial = mat.idMaterial "
                + "WHERE m.idServicio = ?";

            stmt = bd.prepareStatement(sql);
            stmt.setInt(1, idServicio);
            rs = stmt.executeQuery();

            // Configuración del modelo de la tabla.
            TablaMaterialXServicio.setModel(model);

```

```

// Agregar filas al modelo.
while (rs.next()) {
    datos[0] = rs.getString("ServicioCliente");
    String nombreMaterial = rs.getString("nombreMaterial");
    String tipoMaterial = rs.getString("tipoMaterial");
    String material = nombreMaterial + " - " + tipoMaterial;
    datos[1] = material;
    datos[2] = rs.getString("cantMaterial");
    datos[3] = rs.getString("subtotalMaterial") != null ?
rs.getString("subtotalMaterial") : "0.00";
    model.addRow(datos);
}

// Cerrar conexiones.
rs.close();
stmt.close();
bd.close();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar el Material del Servicio:
" + ex.getMessage());
}
}

/*
    Carga la información de la tabla Operativo.Material y la muestra en un
    combobox.
*/
public void CargarMateriales()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos de Postgres.
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
        String sql = "SELECT nombreMaterial, tipoMaterial FROM
Operativo.Material";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();

```

```

// Limpiar el JComboBox antes de agregar datos
cmbboxMaterial.removeAllItems();

// Leer cada domicilio y cliente del resultado de la consulta
while (rs.next()) {
    String nombreMaterial = rs.getString("nombreMaterial");
    String tipoMaterial = rs.getString("tipoMaterial");
    String material = nombreMaterial + " - " + tipoMaterial;

    // Agregar resultado al JComboBox
    cmbboxMaterial.addItem(material);
}
rs.close();
stmt.close();
bd.close();
} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al cargar los materiales.");
}
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Operativo.MaterialXservicio con los datos
ingresados.
*/
private void InsertaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establecer conexión con la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener datos del formulario
        String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();
        String[] partesMaterial = materialSeleccionado.split("\\-");
        String nombreMaterial = partesMaterial[0].trim();
        String tipoMaterial = partesMaterial[1].trim();
        String cant = txtCantidad.getText();
        int cantidad = Integer.parseInt(cant);

```

```

// Obtener idMaterial
long idMaterial = 0;
String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
WHERE NombreMaterial = ? AND TipoMaterial = ?";
stmt = bd.prepareStatement(queryIdMaterial);
stmt.setString(1, nombreMaterial);
stmt.setString(2, tipoMaterial);
rs = stmt.executeQuery();

if (rs.next()) {
    idMaterial = rs.getLong("idMaterial");
} else {
    JOptionPane.showMessageDialog(null, "El material seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

// Verificar si ya existe un servicio programado para este domicilio
String verificarMaterial = "SELECT COUNT(*) FROM
Operativo.MaterialXServicio WHERE idServicio = ? AND idMaterial = ?";
stmt = bd.prepareStatement(verificarMaterial);
stmt.setInt(1, idServicio);
stmt.setLong(2, idMaterial);
rs = stmt.executeQuery();

if (rs.next() && rs.getInt(1) > 0) {
    JOptionPane.showMessageDialog(null, "El material ya está asociado a este
servicio.", "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

// Verificar existencia suficiente en el Material
String verificarExistencia = "SELECT existencia FROM Operativo.Material
WHERE idMaterial = ?";
stmt = bd.prepareStatement(verificarExistencia);
stmt.setLong(1, idMaterial);
rs = stmt.executeQuery();

if (rs.next()) {
    int existenciaActual = rs.getInt("existencia");
    if (existenciaActual < cantidad) {
        JOptionPane.showMessageDialog(null, "No hay suficiente existencia del
material seleccionado.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```

        return;
    }
}

// Insertar el material del servicio
String insertInfo = "INSERT INTO Operativo.MaterialXServicio(idServicio,
idMaterial, cantMaterial) VALUES(?, ?, ?)";
stmt = bd.prepareStatement(insertInfo);
stmt.setInt(1, idServicio);
stmt.setLong(2, idMaterial);
stmt.setInt(3, cantidad);
stmt.executeUpdate();

// Limpiar el comboBox después de la inserción
cmbboxMaterial.setSelectedIndex(-1);
txtCantidad.setText("");

} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al insertar el detalle de la
compra: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
} finally {
    // Cerrar recursos
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Comercial.Compra con los datos
actualizados.
*/
private void ModificaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

```

```

try {
    // Establecer conexión con la base de datos
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

    // Obtener datos del formulario
    String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();
    String[] partesMaterial = materialSeleccionado.split("\\-");
    String nombreMaterial = partesMaterial[0].trim();
    String tipoMaterial = partesMaterial[1].trim();
    String cant = txtCantidad.getText();
    int cantidad = Integer.parseInt(cant);

    // Obtener idMaterial
    long idMaterial = 0;
    String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
WHERE NombreMaterial = ? AND TipoMaterial = ?";
    stmt = bd.prepareStatement(queryIdMaterial);
    stmt.setString(1, nombreMaterial);
    stmt.setString(2, tipoMaterial);
    rs = stmt.executeQuery();

    if (rs.next()) {
        idMaterial = rs.getLong("idMaterial");
    } else {
        JOptionPane.showMessageDialog(null, "El material seleccionado no es válido.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Verificar si ya existe un servicio programado para este domicilio
    String verificarMaterial = "SELECT COUNT(*) FROM
Operativo.MaterialXServicio WHERE idServicio <> ? AND idMaterial = ?";
    stmt = bd.prepareStatement(verificarMaterial);
    stmt.setInt(1, idServicio);
    stmt.setLong(2, idMaterial);
    rs = stmt.executeQuery();

    if (rs.next() && rs.getInt(1) > 0) {
        JOptionPane.showMessageDialog(null, "El material ya está asociado a este servicio.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
}

```

```

        // Verificar existencia suficiente en el Material
        String verificarExistencia = "SELECT existencia FROM Operativo.Material
WHERE idMaterial = ?";
        stmt = bd.prepareStatement(verificarExistencia);
        stmt.setLong(1, idMaterial);
        rs = stmt.executeQuery();

        if (rs.next()) {
            int existenciaActual = rs.getInt("existencia");
            if (existenciaActual < cantidad) {
                JOptionPane.showMessageDialog(null, "No hay suficiente existencia del
material seleccionado.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        // Actualizar el detalle de la compra
        String updateQuery = "UPDATE Operativo.MaterialXServicio SET cantMaterial
= ? WHERE idServicio = ? AND idMaterial = ?";
        stmt = bd.prepareStatement(updateQuery);
        stmt.setInt(1, cantidad);
        stmt.setInt(2, idServicio);
        stmt.setLong(3, idMaterial);
        stmt.executeUpdate();

        // Limpiar el comboBox después de la inserción
        cmbboxMaterial.setSelectedIndex(-1);
        txtCantidad.setText("");

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al modificar el material del
servicio: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        // Cerrar recursos
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

    }

    /*
    EliminaDato
    Elimina el registro seleccionado en la tabla Comercial.DetalleCompra con los
    datos actualizados.
    */
    private void EliminaDato(int idServicio)
    {
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            // Establecer conexión con la base de datos
            bd =
            DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

            // Obtener datos del formulario
            String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();
            String[] partesMaterial = materialSeleccionado.split("\\-");
            String nombreMaterial = partesMaterial[0].trim();
            String tipoMaterial = partesMaterial[1].trim();

            // Obtener idMaterial
            long idMaterial = 0;
            String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
            WHERE NombreMaterial = ? AND TipoMaterial = ?";
            stmt = bd.prepareStatement(queryIdMaterial);
            stmt.setString(1, nombreMaterial);
            stmt.setString(2, tipoMaterial);
            rs = stmt.executeQuery();

            if (rs.next()) {
                idMaterial = rs.getLong("idMaterial");
            } else {
                JOptionPane.showMessageDialog(null, "El material seleccionado no es
                válido.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            // Sentencia SQL para eliminar el material del servicio

```

```
String sql = "DELETE FROM Operativo.MaterialXServicio WHERE idServicio =  
? AND idMaterial = ?";
```

```
stmt = bd.prepareStatement(sql);  
stmt.setInt(1, idServicio);  
stmt.setLong(2, idMaterial);  
stmt.executeUpdate();
```

```
// Limpiar el ComboBox después de la eliminación  
cmbboxMaterial.setSelectedIndex(-1);  
txtCantidad.setText("");
```

```
} catch (SQLException ex) {  
    // Mostrar mensaje de error si algo sale mal  
    JOptionPane.showMessageDialog(null, "Error al eliminar el detalle de la  
compra: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);  
} finally {  
    // Cerrar recursos en el bloque finally  
    try {  
        if (rs != null) rs.close();  
        if (stmt != null) stmt.close();  
        if (bd != null) bd.close();  
    } catch (SQLException e) {  
        JOptionPane.showMessageDialog(null, "Error al cerrar los recursos: " +  
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
}
```

```
/*
```

```
    btnInserta_Click
```

```
    Evento de clic del botón de inserción.
```

```
    Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos.
```

```
*/
```

```
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {  
    InsertaDato(idServicio);  
    ConsultaDatos(idServicio);  
}
```

```
/*
```

```
    btnEliminarActionPerformed
```

```
    Evento de clic del botón de eliminación.
```

```
    Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a  
ConsultaDatos.
```

```
*/
```

```

private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaMaterialXServicio.getSelectedRow();
    if(fila >= 0){
        String valorFila = TablaMaterialXServicio.getValueAt(fila, 0).toString();
        try {
            String[] partes = valorFila.split(" - ");
            idServicio = Integer.parseInt(partes[0].trim());
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Error al procesar el id de la
compra.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        EliminaDato(idServicio);
        ConsultaDatos(idServicio);
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

```

/*
 btnModificarActionPerformed
 Evento de clic del botón de modificación.
 Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
 ConsultaDatos.

```

*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaMaterialXServicio.getSelectedRow();
    if (fila >= 0) {
        String valorFila = TablaMaterialXServicio.getValueAt(fila, 0).toString();
        try {
            String[] partes = valorFila.split(" - ");
            idServicio = Integer.parseInt(partes[0].trim());
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Error al procesar el id de la
compra.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        ModificaDato(idServicio);
        ConsultaDatos(idServicio);
    }
    else {

```

```

        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

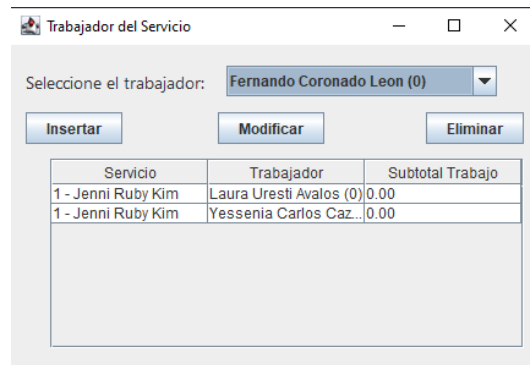
/*
    TablaClienteMouseClicked
    Evento que se activa al hacer clic en una celda del JTable.
    Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
*/
private void TablaMaterialXServicioMouseClicked(java.awt.event.MouseEvent evt)
{
    int fila = TablaMaterialXServicio.getSelectedRow();
    if(fila >= 0) {
        String cantidad = TablaMaterialXServicio.getValueAt(fila, 2).toString();
        txtCantidad.setText(cantidad);

        String material = TablaMaterialXServicio.getValueAt(fila, 1).toString();
        cmbboxMaterial.setSelectedItem(material);
    }
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

public static void main(String args[])
{
    int idServicio = 1;
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new MaterialXServicio(idServicio).setVisible(true);
        }
    });
}

```

TrabajadorXServicio



//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

package Tablas;

// Librerías del sistema usadas para crear y gestionar el formulario TrabajadorXServicio.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
```

/*

Clase para gestionar el manejo de los trabajadores de los servicios dentro de los Servicios de Mantenimiento

Proporciona métodos para consultar, insertar, modificar y eliminar datos.

*/

```
public class TrabajadorXServicio extends javax.swing.JFrame
{
```

```
    int idServicio;
    int idTrabajadorXServicio = -1;
```

/*

Constructor del formulario TrabajadorXServicio.

Inicializa el formulario y carga los datos de la tabla TrabajadorXServicio en el JTable.

*/

```
public TrabajadorXServicio(int idServicio)
{
    initComponents();
    this.idServicio = idServicio;
    ConsultarDatos(idServicio);
    CargarTrabajadores();
```

```

        cmbboxTrabajador.setSelectedIndex(-1);
    }

    /*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla
    Operativo.TrabajadorXServicio en el JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
    */
    private void ConsultaDatos(int idServicio)
    {
        String[] datos = new String[4];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Id Trabajador
del Servicio", "Servicio", "Trabajador", "Subtotal Trabajo"}, 0);
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            // Conexión a la base de datos.
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

            // Consulta SQL con JOINS.
            String sql = "SELECT txs.idTrabajadorXServicio, txs.idServicio,
txs.idTrabajador, txs.subtotalTrabajo, "
                + "CONCAT(t.idServicio, ' - ', c.nombreCliente) AS ServicioCliente, "
                + "CONCAT(tr.nombreTrabajador, ' (', COUNT(dt.idTrabajador), ')') AS
TrabajadorConProfesiones "
                + "FROM Operativo.TrabajadorXServicio txs "
                + "JOIN Operativo.Trabajador tr ON txs.idTrabajador = tr.idTrabajador "
                + "JOIN Operativo.Servicio t ON txs.idServicio = t.idServicio JOIN
Comercial.DomicilioCliente d ON t.idDomicilio = d.idDomicilio "
                + "JOIN Comercial.Cliente c ON d.idCliente = c.idCliente "
                + "LEFT JOIN Operativo.DetalleTrabajador dt ON tr.idTrabajador =
dt.idTrabajador "
                + "WHERE t.idServicio = ? "
                + "GROUP BY txs.idTrabajadorXServicio, txs.idServicio, txs.idTrabajador,
txs.subtotalTrabajo, "
                + "t.idServicio, c.nombreCliente, tr.nombreTrabajador";

            stmt = bd.prepareStatement(sql);

```

```

stmt.setInt(1, idServicio);
rs = stmt.executeQuery();

// Configuración del modelo de la tabla.
TablaTrabajadorXServicio.setModel(model);

// Agregar filas al modelo.
while (rs.next()) {
    datos[0] = rs.getString("idTrabajadorXServicio");
    datos[1] = rs.getString("ServicioCliente");
    datos[2] = rs.getString("TrabajadorConProfesiones");
    datos[3] = rs.getString("subtotalTrabajo") != null ?
rs.getString("subtotalTrabajo") : "0.00";
    model.addRow(datos);
}

TablaTrabajadorXServicio.getColumnModel("Id Trabajador del
Servicio").setMinWidth(0);
TablaTrabajadorXServicio.getColumnModel("Id Trabajador del
Servicio").setMaxWidth(0);
TablaTrabajadorXServicio.getColumnModel("Id Trabajador del Servicio").setWidth(0);

// Cerrar conexiones.
rs.close();
stmt.close();
bd.close();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar el Trabajador del
Servicio: " + ex.getMessage());
}
}

/*
Carge la información de la tabla Operativo.Trabajador y
Operativo.DetalleTrabajador y la muestra en un combobox.
*/
public void CargarTrabajadores()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos de Postgres.

```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
        "postgres");
        String sql = "SELECT t.NombreTrabajador, COUNT(dt.profesion) AS
CantidadProfesiones "
            + "FROM Operativo.Trabajador t "
            + "LEFT JOIN Operativo.DetalleTrabajador dt ON t.idTrabajador =
dt.idTrabajador "
            + "GROUP BY t.NombreTrabajador";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();

        // Limpiar el JComboBox antes de agregar datos
        cmbboxTrabajador.removeAllItems();

        // Leer cada domicilio y cliente del resultado de la consulta
        while (rs.next()) {
            String nombreTrabajador = rs.getString("nombreTrabajador");
            String cantidadProfesiones = rs.getString("cantidadProfesiones");
            String trabajador = nombreTrabajador + " (" + cantidadProfesiones + ")";

            // Agregar resultado al JComboBox
            cmbboxTrabajador.addItem(trabajador);
        }
        rs.close();
        stmt.close();
        bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cargar los trabajadores.");
    }
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Operativo.TrabajadorXServicio con los
datos ingresados.
*/
private void InsertaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

```

```

try
{
    // Obtener datos del formulario
    String trabajadorSeleccionado = cmbboxTrabajador.getSelectedItemAt().toString();
    String[] partesTrabajador = trabajadorSeleccionado.split("\\(");
    String nombreTrabajador = partesTrabajador[0].trim();

    // Establecer conexión con la base de datos
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

    // Obtener idTrabajador
    long idTrabajador = 0;
    String queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador
WHERE nombreTrabajador = ?";
    stmt = bd.prepareStatement(queryIdTrabajador);
    stmt.setString(1, nombreTrabajador);
    rs = stmt.executeQuery();

    if (rs.next()) {
        idTrabajador = rs.getLong("idTrabajador");
    } else {
        JOptionPane.showMessageDialog(null, "El trabajador seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Verificar si ya existe un trabajador asignado a este servicio
    String queryCheckServicio = "SELECT COUNT(*) FROM
Operativo.TrabajadorXServicio WHERE idServicio = ? AND idTrabajador = ?";
    stmt = bd.prepareStatement(queryCheckServicio);
    stmt.setInt(1, idServicio);
    stmt.setLong(2, idTrabajador);
    rs = stmt.executeQuery();

    if (rs.next() && rs.getInt(1) > 0) {
        JOptionPane.showMessageDialog(null, "El trabajador ya está asociado a
este servicio.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Insertar el trabajador al servicio

```

```

        String insertInfo = "INSERT INTO Operativo.TrabajadorXServicio(idServicio,
idTrabajador) VALUES(?, ?)";
        stmt = bd.prepareStatement(insertInfo);
        stmt.setInt(1, idServicio);
        stmt.setLong(2, idTrabajador);
        stmt.executeUpdate();

```

```

// Limpiar el ComboBox después de la inserción
cmbboxTrabajador.setSelectedIndex(-1);

```

```

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al insertar el trabajador en el
servicio: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        // Cerrar recursos
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

/*

```

```

    ModificaDato

```

Modifica el registro seleccionado en la tabla Operativo.TrabajadorXServicio con los datos actualizados.

```

*/

```

```

private void ModificaDato(int Servicio)

```

```

{

```

```

    Connection bd = null;

```

```

    PreparedStatement stmt = null;

```

```

    ResultSet rs = null;

```

```

    try {

```

```

        // Obtener datos del formulario

```

```

        String trabajadorSeleccionado = cmbboxTrabajador.getSelectedItem().toString();

```

```

        String[] partesTrabajador = trabajadorSeleccionado.split("\\(");

```

```

        String nombreTrabajador = partesTrabajador[0].trim();

```

```

        // Establecer conexión con la base de datos

```

```

        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener idTrabajador
        long idTrabajador = 0;
        String queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador WHERE nombreTrabajador = ?";
        stmt = bd.prepareStatement(queryIdTrabajador);
        stmt.setString(1, nombreTrabajador);
        rs = stmt.executeQuery();

        if (rs.next()) {
            idTrabajador = rs.getLong("idTrabajador");
        } else {
            JOptionPane.showMessageDialog(null, "El trabajador seleccionado no es válido.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Verificar si ya existe un trabajador asignado a este servicio
        String queryCheckServicio = "SELECT COUNT(*) FROM Operativo.TrabajadorXServicio WHERE idServicio = ? AND idTrabajador = ?";
        stmt = bd.prepareStatement(queryCheckServicio);
        stmt.setInt(1, idServicio);
        stmt.setLong(2, idTrabajador);
        rs = stmt.executeQuery();

        if (rs.next() && rs.getInt(1) > 0) {
            JOptionPane.showMessageDialog(null, "El trabajador ya está asociado a este servicio.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Actualizar el detalle de la compra
        String updateQuery = "UPDATE Operativo.TrabajadorXServicio SET idTrabajador = ? WHERE idTrabajadorXServicio = ?";
        stmt = bd.prepareStatement(updateQuery);
        stmt.setLong(1, idTrabajador);
        stmt.setInt(2, idTrabajadorXServicio);
        stmt.executeUpdate();

        // Limpiar el comboBox después de la inserción
        cmbboxTrabajador.setSelectedIndex(-1);

```

```

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al modificar el trabajador del
servicio: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        // Cerrar recursos
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
    EliminaDato
    Elimina el registro seleccionado en la tabla Operativo.Servicio con los datos
actualizados.
*/
private void EliminaDato(int idServicio)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try
    {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimi
ento", "postgres",
"postgres");
        String sql = "DELETE FROM Operativo.TrabajadorXServicio WHERE
idTrabajadorXServicio = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idTrabajadorXServicio);

        stmt.executeUpdate();

        cmbboxTrabajador.setSelectedIndex(-1);
        stmt.close();
        bd.close();
    } catch (SQLException ex){

```

```

        JOptionPane.showMessageDialog(null, "Error al eliminar el Servicio.");
    }
}

private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    InsertaDato(idServicio);
    ConsultaDatos(idServicio);
}

private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaTrabajadorXServicio.getSelectedRow();
    if (fila >= 0) {
        idTrabajadorXServicio =
Integer.parseInt(TablaTrabajadorXServicio.getValueAt(fila, 0).toString());
        EliminaDato(idServicio);
        ConsultaDatos(idServicio);

    } else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaTrabajadorXServicio.getSelectedRow();
    if (fila >= 0) {
        idTrabajadorXServicio =
Integer.parseInt(TablaTrabajadorXServicio.getValueAt(fila, 0).toString());
        ModificaDato(idServicio);
        ConsultaDatos(idServicio);

    } else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

private void TablaTrabajadorXServicioMouseClicked(java.awt.event.MouseEvent
evt) {
    int fila = TablaTrabajadorXServicio.getSelectedRow();
    if(fila >= 0) {
        String trabajador = TablaTrabajadorXServicio.getValueAt(fila, 2).toString();
        cmbboxTrabajador.setSelectedItem(trabajador);
    }
}

```

```

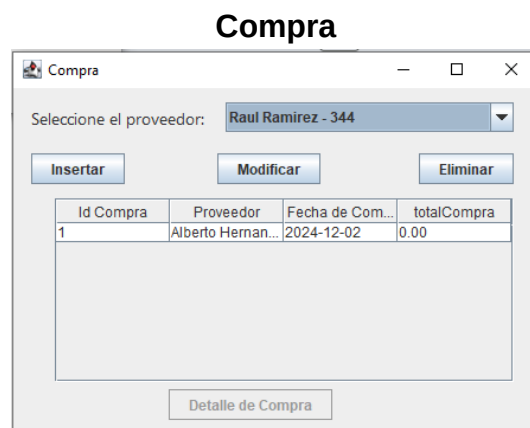
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

```

```

public static void main(String args[]) {
    int idServicio = 1;
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new TrabajadorXServicio(idServicio).setVisible(true);
        }
    });
}

```



//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Compra.

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.util.Date;
import javax.swing.JComboBox;
import com.toedter.calendar.JDateChooser;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.List;

```

```

import java.math.BigDecimal;

/*
    Clase para gestionar el manejo de las compras dentro de los Servicios de
    Mantenimiento
    Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class Compra extends javax.swing.JFrame
{
    private String usuario;
    private String rol;
    int idCompra = -1;

    // Lista que almacena objetos de tipo DetalleCompra.
    private List<DetalleCompra> detalleCompraForms = new
    ArrayList<DetalleCompra>();

    /*
        Constructor del formulario Compra.
        Inicializa el formulario y carga los datos de la tabla Compra en el JTable.
    */
    public Compra(String usuario, String rol)
    {
        this.usuario = usuario;
        this.rol = rol;
        initComponents();
        ConsultaDatos();
        cargarProveedores();
        cmbboxProveedor.setSelectedIndex(-1);
        configurarPermisos();
    }

    /*
        configurarPermisos
        Permite deshabilitar los botones de Insertar, Modificar y Eliminar para el
        usuario Empleado
    */
    public void configurarPermisos()
    {
        if(rol.equals("Empleado")){
            btnInsertar.setEnabled(false);
            btnModificar.setEnabled(false);
            btnEliminar.setEnabled(false);
        }
    }
}

```

```

    }

    /*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Comercial.Cliente en el
    JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
    */
    private void ConsultaDatos()
    {

        String[] datos = new String[4];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Id Compra",
        "Proveedor", "Fecha de Compra", "totalCompra"}, 0);
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;
        try
        {
            // Conexión a la base de datos de Postgres.
            bd =
            DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
            "postgres");
            String sql = "SELECT c.idCompra, c.idProveedor, c.fechaCompra,
            c.totalCompra, " +
            "p.NombreProveedor, p.TelefonoProveedor " +
            "FROM Comercial.Compra c " +
            "JOIN Comercial.Proveedor p ON c.idProveedor = p.idProveedor";
            stmt = bd.prepareStatement(sql);
            rs = stmt.executeQuery();
            TablaCompra.setModel(model);

            // Agrega los valores a una nueva fila para almacenarlos en el JTable.
            while(rs.next()){
                String nombreProveedor = rs.getString("nombreProveedor");
                String telefonoProveedor = rs.getString("telefonoProveedor");
                String domicilio = nombreProveedor + " - " +
                telefonoProveedor.substring(telefonoProveedor.length()-3);
                datos[0] = String.valueOf(rs.getInt("idCompra"));
                datos[1] = domicilio;
                datos[2] = rs.getString("fechaCompra");
                BigDecimal totalCompra = rs.getBigDecimal("totalCompra");
            }
        }
    }

```

```

        datos[3] = (totalCompra != null) ? totalCompra.toString() : "0.00";
        model.addRow(datos);
    }
    rs.close();
    stmt.close();
    bd.close();
} catch (SQLException ex){
    JOptionPane.showMessageDialog(null, "Error al cargar la Compra.");
}
}

/*
    Carga la información de la tabla Comercial.Proveedor y la muestra en un
    combobox.
*/
public void cargarProveedores()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos de Postgres.
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
        "postgres");
        String sql = "SELECT nombreProveedor, telefonoProveedor FROM
        Comercial.Proveedor";
        stmt = bd.prepareStatement(sql);
        rs = stmt.executeQuery();

        // Limpiar el JComboBox antes de agregar datos
        cmbboxProveedor.removeAllItems();

        // Leer cada domicilio y cliente del resultado de la consulta
        while (rs.next()) {
            String nombreProveedor = rs.getString("nombreProveedor");
            String telefonoProveedor = rs.getString("telefonoProveedor");
            String domicilio = nombreProveedor + " - " +
            telefonoProveedor.substring(telefonoProveedor.length()-3);

            // Agregar resultado al JComboBox
            cmbboxProveedor.addItem(domicilio);
        }
    }
}

```

```

    }
    rs.close();
    stmt.close();
    bd.close();
} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al cargar los domicilios.");
}
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Comercial.Compra con los datos
ingresados.
*/
public void InsertaDato()
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Obtener datos del formulario
        String proveedorSeleccionado=cmboxProveedor.getSelectedItemAt().toString();
        String[] partesProveedor = proveedorSeleccionado.split("\\-");
        String nombreProveedor = partesProveedor[0].trim();
        String ultimosTresTelefono = partesProveedor[1].trim();

        java.sql.Date fechaCompra = new java.sql.Date(new Date().getTime());

        // Conexión a la base de datos de Postgres.
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener idProveedor
        long idProveedor = 0;
        String queryIdProveedor = "SELECT idProveedor FROM Comercial.Proveedor
WHERE nombreProveedor = ? AND RIGHT(telefonoProveedor, 3) = ?";
        stmt = bd.prepareStatement(queryIdProveedor);
        stmt.setString(1, nombreProveedor);
        stmt.setString(2, ultimosTresTelefono);
        rs = stmt.executeQuery();

        if (rs.next()) {

```

```

        idProveedor = rs.getLong("idProveedor");
    } else {
        JOptionPane.showMessageDialog(null, "El proveedor seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    String insertInfo = "INSERT INTO Comercial.Compra(idProveedor,
fechaCompra)VALUES(?, ?)";
    stmt = bd.prepareStatement(insertInfo);
    stmt.setLong(1, idProveedor);
    stmt.setDate(2, fechaCompra);
    stmt.executeUpdate();

    cmbboxProveedor.setSelectedIndex(-1);

} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al insertar la compra: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
} finally {
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Comercial.Proveedor con los datos
actualizados.
*/
public void ModificaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Obtener datos del formulario
        String proveedorSeleccionado = (String)cmbboxProveedor.getSelectedItem();

```

```

        if (proveedorSeleccionado == null || proveedorSeleccionado.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Seleccione un proveedor válido.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        String[] partesProveedor = proveedorSeleccionado.split("\\-");
        String nombreProveedor = partesProveedor[0].trim();
        String ultimosTresTelefono = partesProveedor[1].trim();

        java.sql.Date fechaCompra = new java.sql.Date(new Date().getTime());

        // Conexión a la base de datos de Postgres.
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener idCompra
        long idProveedor = 0; // Cambiar a tipo long
        String queryIdProveedor = "SELECT idProveedor FROM Comercial.Proveedor
WHERE nombreProveedor = ? AND RIGHT(telefonoProveedor, 3) = ?";
        stmt = bd.prepareStatement(queryIdProveedor);
        stmt.setString(1, nombreProveedor);
        stmt.setString(2, ultimosTresTelefono);
        rs = stmt.executeQuery();

        if (rs.next()) {
            idProveedor = rs.getLong("idProveedor"); // Usar getLong en lugar de
getString
        } else {
            JOptionPane.showMessageDialog(null, "El proveedor seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        String insertInfo = "UPDATE Comercial.Compra SET idProveedor = ?,
fechaCompra = ? WHERE idCompra = ?";
        stmt = bd.prepareStatement(insertInfo);
        stmt.setLong(1, idProveedor);
        stmt.setDate(2, fechaCompra);
        stmt.setInt(3, idCompra);
        stmt.executeUpdate();

        cmbboxProveedor.setSelectedIndex(-1);

    }catch (SQLException e) {

```

```

        JOptionPane.showMessageDialog(null, "Error al insertar el servicio: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

EliminaDato

Elimina el registro seleccionado en la tabla Comercial.Compra con los datos actualizados.

```

private void EliminaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;

    try
    {
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
        String sql = "DELETE FROM Comercial.Compra WHERE idCompra = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idCompra);

        stmt.executeUpdate();

        cmbboxProveedor.setSelectedIndex(-1);
        stmt.close();
        bd.close();
    } catch (SQLException ex){
        JOptionPane.showMessageDialog(null, "Error al eliminar la Compra.");
    }
}

```

SeleccionaDato

Selecciona los datos de la compra y los muestra en los campos de texto correspondientes.

```
public void SeleccionaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Conexión a la base de datos de Postgres.
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Consulta para obtener los datos de la compra según idCompra
        String sql = "SELECT c.idCompra, c.idProveedor, c.fechaCompra, c.totalCompra, " +
            "p.NombreProveedor, p.TelefonoProveedor " +
            "FROM Comercial.Compra c " +
            "JOIN Comercial.Proveedor p ON c.idProveedor = p.idProveedor " +
            "WHERE c.idCompra = ?";
        stmt = bd.prepareStatement(sql);
        stmt.setInt(1, idCompra);
        rs = stmt.executeQuery();

        // Verificamos si se encontró la compra
        if (rs.next()) {
            String nombreProveedor = rs.getString("NombreProveedor");
            String telefonoProveedor = rs.getString("TelefonoProveedor");
            String domicilio = nombreProveedor + " - " +
            telefonoProveedor.substring(telefonoProveedor.length() - 3);

            // Asignar el valor al ComboBox
            cmbboxProveedor.setSelectedItem(domicilio); // Asumimos que
            cmbboxProveedor es el JComboBox

        } else {
            JOptionPane.showMessageDialog(null, "No se encontró la compra con el ID especificado.", "Error", JOptionPane.ERROR_MESSAGE);
        }

    } catch (SQLException e) {
```

```
JOptionPane.showMessageDialog(null, "Error al seleccionar los datos de la compra: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
```

```
} finally {
```

```
    try {
```

```
        if (rs != null) rs.close();
```

```
        if (stmt != null) stmt.close();
```

```
        if (bd != null) bd.close();
```

```
    } catch (SQLException e) {
```

```
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
```

```
    }
```

```
}
```

```
}
```

```
/*
```

```
    btnInsertarActionPerformed
```

```
    Evento de clic del botón de inserción.
```

```
    Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los campos de entrada.
```

```
*/
```

```
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
```

```
    InsertaDato();
```

```
    ConsultaDatos();
```

```
}
```

```
/*
```

```
    btnEliminarActionPerformed
```

```
    Evento de clic del botón de eliminación.
```

```
    Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a ConsultaDatos.
```

```
*/
```

```
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
```

```
    int fila = TablaCompra.getSelectedRow();
```

```
    if(fila >= 0){
```

```
        idCompra = Integer.parseInt(TablaCompra.getValueAt(fila, 0).toString());
```

```
        EliminaDato(idCompra);
```

```
        ConsultaDatos();
```

```
    }
```

```
    else{
```

```
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila válida.");
```

```
    }
```

```
}
```

```

/*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
    ConsultaDatos.
*/
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = TablaCompra.getSelectedRow();
    if(fila >= 0){
        idCompra = Integer.parseInt(TablaCompra.getValueAt(fila, 0).toString());
        ModificaDato(idCompra);
        ConsultaDatos();
    }
    else{
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    TablaClienteMouseClicked
    Evento que se activa al hacer clic en una celda del JTable.
    Llama al método SeleccionaDato para cargar los datos del cliente en los
    campos de texto.
*/
private void TablaCompraMouseClicked(java.awt.event.MouseEvent evt) {
    int fila = TablaCompra.getSelectedRow();
    if(fila >= 0) {
        idCompra = Integer.parseInt(TablaCompra.getValueAt(fila, 0).toString());
        SeleccionaDato(idCompra);

        if(!rol.equals("Empleado")){
            btnDetalleCompra.setEnabled(true);
        }
    }
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

/*
    btnDetalleCompraActionPerformed
    Evento que se dispara al hacer clic en el botón de Detalle de Compra.

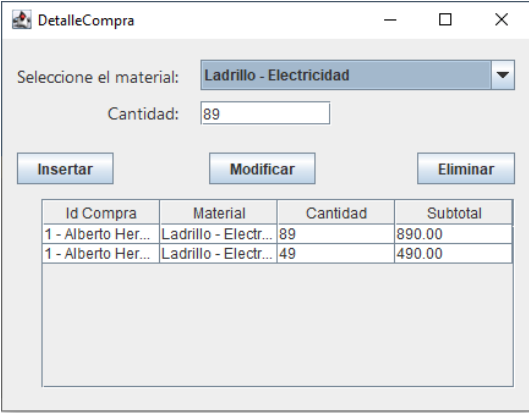
```

Abre el formulario DetalleCompra para gestionar los detalles asociados a una compra específica.

```
*/
private void btnDetalleCompraActionPerformed(java.awt.event.ActionEvent evt) {
    if(idCompra != -1)
    {
        DetalleCompra detalleCompraForm = new DetalleCompra(idCompra);
        detalleCompraForms.add(detalleCompraForm);
        detalleCompraForm.addWindowListener(new
java.awt.event.WindowAdapter() {
            public void windowClosing(java.awt.event.WindowEvent e) {
                detalleCompraForms.remove(detalleCompraForm);
            }
        });
        detalleCompraForm.setVisible(true);
    }
}

public static void main(String args[]) {
    String usuario = "";
    String rol = "";
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Compra(usuario, rol).setVisible(true);
        }
    });
}
```

Detalle Compra



Id Compra	Material	Cantidad	Subtotal
1 - Alberto Her...	Ladrillo - Electr...	89	890.00
1 - Alberto Her...	Ladrillo - Electr...	49	490.00

// Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario DetalleCompra

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/*
    Clase para gestionar el manejo de los detalles de las compras dentro de los
    Servicios de Mantenimiento
    Proporciona métodos para consultar, insertar, modificar y eliminar datos.
*/
public class DetalleCompra extends javax.swing.JFrame
{
    int idCompra;

    /*
        Constructor del formulario DetalleCompra.
        Inicializa el formulario y carga los datos de la tabla DetalleCompra en el
        DataGridView.
    */
    public DetalleCompra(int idCompra)
    {
        initComponents();
        this.idCompra = idCompra;
        ConsultaDatos(idCompra);
        cargarMateriales();
        cmbboxMaterial.setSelectedIndex(-1);
    }

    /*
        ConsultaDatos
        Recupera y muestra todos los registros de la tabla Comercial.Cliente en el
        JTable.
        Creación del modelo de la tabla agregando los encabezados de cada columna
        donde inicialmente no contiene filas.
    */
    private void ConsultaDatos(int idCompra)
    {
        String[] datos = new String[4];
        DefaultTableModel model = new DefaultTableModel(new String[]{"Id Compra",
        "Material", "Cantidad", "Subtotal"}, 0);
        Connection bd = null;
    }

```

```

PreparedStatement stmt = null;
ResultSet rs = null;

try {
    // Conexión a la base de datos.
    bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

    // Consulta SQL con JOINS.
    String sql = "SELECT " +
        "CONCAT(d.idCompra, ' - ', p.nombreProveedor) AS idCompra, " +
        "CONCAT(m.NombreMaterial, ' - ', m.TipoMaterial) AS Material, " +
        "d.cantidad, CAST(d.subtotal AS numeric(10, 2)) AS subtotal " +
        "FROM Comercial.DetalleCompra d " +
        "JOIN Comercial.Compra c ON d.idCompra = c.idCompra " +
        "JOIN Comercial.Proveedor p ON c.idProveedor = p.idProveedor " +
        "JOIN Operativo.Material m ON d.idMaterial = m.idMaterial " +
        "WHERE d.idCompra = ?";

    stmt = bd.prepareStatement(sql);
    stmt.setInt(1, idCompra); // Establecer parámetro.
    rs = stmt.executeQuery();

    // Configuración del modelo de la tabla.
    TablaDetalleCompra.setModel(model);

    // Agregar filas al modelo.
    while (rs.next()) {
        datos[0] = rs.getString("idCompra");
        datos[1] = rs.getString("Material");
        datos[2] = String.valueOf(rs.getInt("cantidad"));
        datos[3] = String.format("%.2f", rs.getFloat("subtotal"));
        model.addRow(datos);
    }

    // Cerrar conexiones.
    rs.close();
    stmt.close();
    bd.close();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar el Detalle de la Compra: " + ex.getMessage());
}

```

```

    }

    /*
        Carga la información de la tabla Operativo.Material y la muestra en un
        combobox.
    */
    public void cargarMateriales()
    {
        Connection bd = null;
        PreparedStatement stmt = null;
        ResultSet rs = null;

        try {
            // Conexión a la base de datos de Postgres.
            bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres",
"postgres");
            String sql = "SELECT nombreMaterial, tipoMaterial FROM
Operativo.Material";
            stmt = bd.prepareStatement(sql);
            rs = stmt.executeQuery();

            // Limpiar el JComboBox antes de agregar datos
            cmbboxMaterial.removeAllItems();

            // Leer cada domicilio y cliente del resultado de la consulta
            while (rs.next()) {
                String nombreMaterial = rs.getString("nombreMaterial");
                String tipoMaterial = rs.getString("tipoMaterial");
                String material = nombreMaterial + " - " + tipoMaterial;

                // Agregar resultado al JComboBox
                cmbboxMaterial.addItem(material);
            }
            rs.close();
            stmt.close();
            bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cargar los materiales.");
        }
    }

    /*

```

InsertaDato

Inserta un nuevo registro en la tabla Comercial.DetalleCompra con los datos ingresados.

```
*/
private void InsertaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establecer conexión con la base de datos
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener datos del formulario
        String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();
        String[] partesMaterial = materialSeleccionado.split("\\-");
        String nombreMaterial = partesMaterial[0].trim();
        String tipoMaterial = partesMaterial[1].trim();
        String cant = txtCantidad.getText();
        int cantidad = Integer.parseInt(cant);

        // Obtener idMaterial
        long idMaterial = 0;
        String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
WHERE NombreMaterial = ? AND TipoMaterial = ?";
        stmt = bd.prepareStatement(queryIdMaterial);
        stmt.setString(1, nombreMaterial);
        stmt.setString(2, tipoMaterial);
        rs = stmt.executeQuery();

        if (rs.next()) {
            idMaterial = rs.getLong("idMaterial");
        } else {
            JOptionPane.showMessageDialog(null, "El material seleccionado no es válido.", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Insertar el detalle de la compra
        String insertInfo = "INSERT INTO Comercial.DetalleCompra(idCompra, idMaterial, cantidad) VALUES(?, ?, ?)";
```

```

        stmt = bd.prepareStatement(insertInfo);
        stmt.setInt(1, idCompra);
        stmt.setLong(2, idMaterial);
        stmt.setInt(3, cantidad);
        stmt.executeUpdate();

        // Limpiar el comboBox después de la inserción
        cmbboxMaterial.setSelectedIndex(-1);

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al insertar el detalle de la
        compra: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    } finally {
        // Cerrar recursos
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (bd != null) bd.close();
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
            e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/*
    ModificaDato
    Modifica el registro seleccionado en la tabla Comercial.Compra con los datos
    actualizados.
    */
private void ModificaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establecer conexión con la base de datos
        bd =
        DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener datos del formulario
        String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();

```

```

String[] partesMaterial = materialSeleccionado.split("\\-");
String nombreMaterial = partesMaterial[0].trim();
String tipoMaterial = partesMaterial[1].trim();
String cant = txtCantidad.getText();
int cantidad = Integer.parseInt(cant);

// Obtener idMaterial
long idMaterial = 0;
String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
WHERE NombreMaterial = ? AND TipoMaterial = ?";
stmt = bd.prepareStatement(queryIdMaterial);
stmt.setString(1, nombreMaterial);
stmt.setString(2, tipoMaterial);
rs = stmt.executeQuery();

if (rs.next()) {
    idMaterial = rs.getLong("idMaterial");
} else {
    JOptionPane.showMessageDialog(null, "El material seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

// Actualizar el detalle de la compra
String updateQuery = "UPDATE Comercial.DetalleCompra SET cantidad = ?
WHERE idCompra = ? AND idMaterial = ?";
stmt = bd.prepareStatement(updateQuery);
stmt.setInt(1, cantidad); // Asignar la cantidad
stmt.setInt(2, idCompra); // Asignar idCompra
stmt.setLong(3, idMaterial); // Asignar idMaterial
stmt.executeUpdate();

// Limpiar el comboBox después de la inserción
comboBoxMaterial.setSelectedIndex(-1);
txtCantidad.setText("");

} catch (SQLException e) {
    JOptionPane.showMessageDialog(null, "Error al modificar el detalle de la
compra: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
} finally {
    // Cerrar recursos
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
    }
}

```

```

        if (bd != null) bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Comercial.DetalleCompra con los
datos actualizados.
*/
private void EliminaDato(int idCompra)
{
    Connection bd = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;

    try {
        // Establecer conexión con la base de datos
        bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");

        // Obtener datos del formulario
        String materialSeleccionado = cmbboxMaterial.getSelectedItem().toString();
        String[] partesMaterial = materialSeleccionado.split("\\-");
        String nombreMaterial = partesMaterial[0].trim();
        String tipoMaterial = partesMaterial[1].trim();

        // Obtener idMaterial
        long idMaterial = 0;
        String queryIdMaterial = "SELECT idMaterial FROM Operativo.Material
WHERE NombreMaterial = ? AND TipoMaterial = ?";
        stmt = bd.prepareStatement(queryIdMaterial);
        stmt.setString(1, nombreMaterial);
        stmt.setString(2, tipoMaterial);
        rs = stmt.executeQuery();

        if (rs.next()) {
            idMaterial = rs.getLong("idMaterial");
        } else {

```

```

        JOptionPane.showMessageDialog(null, "El material seleccionado no es
válido.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Sentencia SQL para eliminar el detalle de la compra
    String sql = "DELETE FROM Comercial.DetalleCompra WHERE idCompra = ?
AND idMaterial = ?";
    stmt = bd.prepareStatement(sql);
    stmt.setInt(1, idCompra); // Aquí pasa el idCompra (debe ser un entero)
    stmt.setLong(2, idMaterial); // Aquí pasa el idMaterial obtenido de la consulta

    // Ejecutar la eliminación
    int rowsAffected = stmt.executeUpdate();

    // Limpiar el ComboBox después de la eliminación
    cmbboxMaterial.setSelectedIndex(-1);

} catch (SQLException ex) {
    // Mostrar mensaje de error si algo sale mal
    JOptionPane.showMessageDialog(null, "Error al eliminar el detalle de la
compra: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
} finally {
    // Cerrar recursos en el bloque finally
    try {
        if (rs != null) rs.close();
        if (stmt != null) stmt.close();
        if (bd != null) bd.close();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(null, "Error al cerrar los recursos: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}

/*
    btnInsertarActionPerformed
    Evento de clic del botón de inserción.
    Llama a InsertaDato, actualiza el JTable llamando a ConsultaDatos y limpia los
campos de entrada.
*/
private void btnInsertarActionPerformed(java.awt.event.ActionEvent evt) {
    InsertaDato(idCompra);
    ConsultaDatos(idCompra);
}

```

```

    }

    /*
    btnEliminarActionPerformed
    Evento de clic del botón de eliminación.
    Extrae el idCliente, llama a EliminaDato, actualiza el JTable llamando a
    ConsultaDatos.
    */
    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaDetalleCompra.getSelectedRow();
        if(fila >= 0){
            String valorFila = TablaDetalleCompra.getValueAt(fila, 0).toString();
            try {
                String[] partes = valorFila.split(" - ");
                idCompra = Integer.parseInt(partes[0].trim());
            } catch (Exception e) {
                JOptionPane.showMessageDialog(null, "Error al procesar el id de la
                compra.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
            EliminaDato(idCompra);
            ConsultaDatos(idCompra);
        }
        else{
            JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
            válida.");
        }
    }

    /*
    btnModificarActionPerformed
    Evento de clic del botón de modificación.
    Extrae el idCliente, llama a ModificaDato, actualiza el JTable llamando a
    ConsultaDatos.
    */
    private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {
        int fila = TablaDetalleCompra.getSelectedRow();
        if (fila >= 0) {
            String valorFila = TablaDetalleCompra.getValueAt(fila, 0).toString();
            try {
                String[] partes = valorFila.split(" - ");
                idCompra = Integer.parseInt(partes[0].trim());
            } catch (Exception e) {

```

```

        JOptionPane.showMessageDialog(null, "Error al procesar el id de la
compra.", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    ModificaDato(idCompra);
    ConsultaDatos(idCompra);

} else {
    JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila válida.");
}
}

/*
    TablaClienteMouseClicked
    Evento que se activa al hacer clic en una celda del JTable.
    Llama al método SeleccionaDato para cargar los datos del cliente en los
campos de texto.
*/
private void TablaDetalleCompraMouseClicked(java.awt.event.MouseEvent evt) {
    int fila = TablaDetalleCompra.getSelectedRow();
    if(fila >= 0) {
        String cantidad = TablaDetalleCompra.getValueAt(fila, 2).toString();
        txtCantidad.setText(cantidad);

        String material = TablaDetalleCompra.getValueAt(fila, 1).toString();
        cmbboxMaterial.setSelectedItem(material);
    }
    else {
        JOptionPane.showMessageDialog(null, "Por favor, selecciona una fila
válida.");
    }
}

public static void main(String args[]) {
    int idCompra = 1;
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new DetalleCompra(idCompra).setVisible(true);
        }
    });
}
}

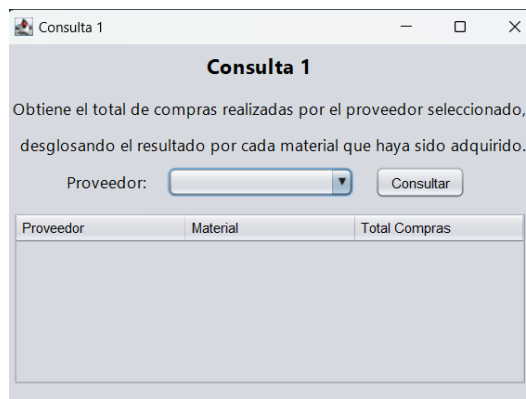
```

Consultas

Para ingresar a Consultas en el login de usuario entraremos como admin y la contraseña AdminPostgres, nos aparecerá el siguiente menú, aparecen dos botones en la parte de abajo llamado Consulta1 y Consulta2.



Para la Consulta 1



//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

package Tablas;

// Librerías del sistema usadas para crear y gestionar el formulario Cliente

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
```

/*

Clase para gestionar el manejo de la Consulta 1 en Servicios de Mantenimiento

a) Consulta 1: Uso de GROUP BY, COUNT e INNER JOIN

*/

```
public class Consulta1 extends javax.swing.JFrame {
    private String nombreProveedor = "";
```

/**

* Constructor del formulario Consulta1.

```

    * Inicializa el formulario.
    */
    public Consulta1()
    {
        initComponents();
        cargarProveedores();
        consultaTabla();
        cmbConsulta1.setSelectedIndex(-1);
    }

    /*
    consulta
    Recupera y muestra todos los registros en el JTable.
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
    */
    private void consulta()
    {
        nombreProveedor = cmbConsulta1.getSelectedItem().toString();

        String[] columnas = {"Proveedor", "Material", "Total Compras"};
        DefaultTableModel model = new DefaultTableModel(columnas, 0);

        String sql = "SELECT p.nombreProveedor AS NombreProveedor, "
            + "m.nombreMaterial AS NombreMaterial, "
            + "COUNT(c.idCompra) AS TotalCompras "
            + "FROM Comercial.Proveedor p "
            + "INNER JOIN Comercial.Compra c ON p.idProveedor = c.idProveedor
"
            + "INNER JOIN Comercial.DetalleCompra dc ON c.idCompra =
dc.idCompra "
            + "INNER JOIN Operativo.Material m ON dc.idMaterial = m.idMaterial "
            + "WHERE p.nombreProveedor = ? "
            + "GROUP BY p.nombreProveedor, m.nombreMaterial";

        try (Connection bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        PreparedStatement stmt = bd.prepareStatement(sql)) {

            stmt.setString(1, nombreProveedor);
            try (ResultSet rs = stmt.executeQuery())
            {
                TablaConsulta.setModel(model);
            }
        }
    }

```

```

// Agrega los valores a una nueva fila para almacenarlos en el JTable
while (rs.next()) {
    model.addRow(new Object[]{
        rs.getString("NombreProveedor"),
        rs.getString("NombreMaterial"),
        rs.getInt("TotalCompras")
    });
}
}
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar los datos: " +
ex.getMessage());
}
}

/*
    consultaTabla
    Creación del modelo de la tabla agregando los encabezados de cada columna
    donde inicialmente no contiene filas.
*/
private void consultaTabla()
{
    nombreProveedor = "";

    String[] columnas = {"Proveedor", "Material", "Total Compras"};
    DefaultTableModel model = new DefaultTableModel(columnas, 0);

    String sql = "SELECT p.nombreProveedor AS NombreProveedor, "
        + "m.nombreMaterial AS NombreMaterial, "
        + "COUNT(c.idCompra) AS TotalCompras "
        + "FROM Comercial.Proveedor p "
        + "INNER JOIN Comercial.Compra c ON p.idProveedor = c.idProveedor "
        + "INNER JOIN Comercial.DetalleCompra dc ON c.idCompra = "
        + "dc.idCompra "
        + "INNER JOIN Operativo.Material m ON dc.idMaterial = m.idMaterial "
        + "WHERE p.nombreProveedor = ? "
        + "GROUP BY p.nombreProveedor, m.nombreMaterial";

    try (Connection bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        PreparedStatement stmt = bd.prepareStatement(sql)) {

```

```

        stmt.setString(1, nombreProveedor);
        try (ResultSet rs = stmt.executeQuery())
        {
            TablaConsulta.setModel(model);
            // Agrega los valores a una nueva fila para almacenarlos en el JTable
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getString("NombreProveedor"),
                    rs.getString("NombreMaterial"),
                    rs.getInt("TotalCompras")
                });
            }
        }
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al cargar los datos: " +
ex.getMessage());
    }
}

/**
 * Carga la información de los proveedores desde la base de datos y la muestra
en un JComboBox.
 */
public void cargarProveedores()
{
    String sql = "SELECT nombreProveedor FROM Comercial.Proveedor";

    try (Connection bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        PreparedStatement stmt = bd.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        // Limpiar el JComboBox antes de agregar datos
        cmbConsulta1.removeAllItems();

        // Leer cada proveedor del resultado de la consulta
        while (rs.next()) {
            cmbConsulta1.addItem(rs.getString("nombreProveedor"));
        }

    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(null, "Error al cargar los proveedores: " +
ex.getMessage());
    }
}

```

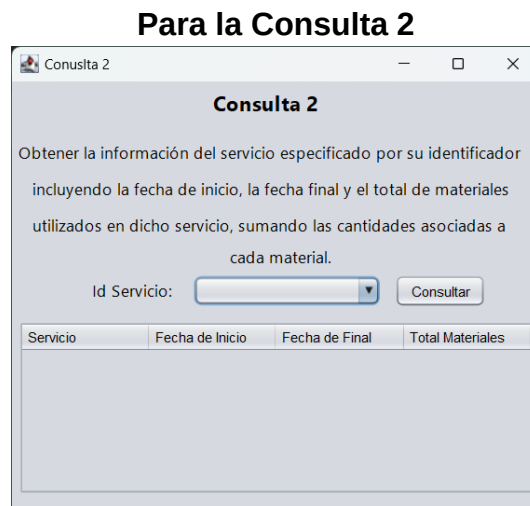
```

    }
}

/*
    btnConsultaActionPerformed
    Evento de clic del botón de Consulta.
    Llama al método para realizar la consulta.
*/
private void btnConsultaActionPerformed(java.awt.event.ActionEvent evt) {
    consulta();
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Consulta1().setVisible(true);
        }
    });
}
}

```



//Conjunto de clases en la cual contendrá las tablas del proyecto Servicios de Mantenimiento.

```
package Tablas;
```

// Librerías del sistema usadas para crear y gestionar el formulario Cliente

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

```

```

import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/*
    Clase para gestionar el manejo de la Consulta 1 en Servicios de Mantenimiento
    b) Consulta 2: Realizar subconsultas
*/
public class Consulta2 extends javax.swing.JFrame
{
    /*
        Constructor del formulario Consulta2.
        Inicializa el formulario.
    */
    public Consulta2() {
        initComponents();
        cargarServicios();
        consultaTabla();
        cmbConsulta1.setSelectedIndex(-1);
    }

    /*
        consulta
        Recupera y muestra todos los registros en el JTable.
        Creación del modelo de la tabla agregando los encabezados de cada columna
        donde inicialmente no contiene filas.
    */
    private void consulta()
    {
        String servicioSeleccionado = cmbConsulta1.getSelectedItem().toString();
        String idServicio = servicioSeleccionado.split(" - ")[0].trim();
        int servicio = Integer.parseInt(idServicio);

        String[] columnas = {"Servicio", "Fecha de Inicio", "Fecha de Final", "Total
        Materiales"};
        DefaultTableModel model = new DefaultTableModel(columnas, 0);

        String sql = "SELECT s.idServicio, s.fechaInicio, s.fechaFinal, " +
            "CONCAT(s.idServicio, ' - ', c.nombreCliente) AS ServicioCliente, " +
            "COALESCE((SELECT SUM(mx.cantMaterial) " +
            "FROM Operativo.MaterialXServicio mx WHERE mx.idServicio =
            s.idServicio), 0) AS TotalMateriales " +
            "FROM Operativo.Servicio s " +
            "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio
            " +

```

```
"JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente " +  
"WHERE s.idServicio = ?";
```

```
try (Connection bd =  
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");  
    PreparedStatement stmt = bd.prepareStatement(sql)) {  
  
    stmt.setInt(1, servicio);  
  
    try (ResultSet rs = stmt.executeQuery())  
    {  
        TablaConsulta.setModel(model);  
        // Agrega los valores a una nueva fila para almacenarlos en el JTable  
        while (rs.next()) {  
            model.addRow(new Object[]{  
                rs.getString("ServicioCliente"),  
                rs.getString("fechaInicio"),  
                rs.getString("fechaFinal"),  
                rs.getInt("TotalMateriales")  
            });  
        }  
    }  
  
    } catch (SQLException ex) {  
        JOptionPane.showMessageDialog(null, "Error al cargar los datos: " +  
ex.getMessage());  
    }  
}  
  
/*  
    consultaTabla  
    Creación del modelo de la tabla agregando los encabezados de cada columna  
    donde inicialmente no contiene filas.  
*/  
private void consultaTabla()  
{  
    int servicio = -1;  
    String[] columnas = {"Servicio", "Fecha de Inicio", "Fecha de Final", "Total  
Materiales"};  
    DefaultTableModel model = new DefaultTableModel(columnas, 0);  
    String sql = "SELECT s.idServicio, s.fechaInicio, s.fechaFinal, " +  
        "CONCAT(s.idServicio, ' - ', c.nombreCliente) AS ServicioCliente, " +  
        "COALESCE((SELECT SUM(mx.cantMaterial) " +
```

```

        "FROM Operativo.MaterialXServicio mx WHERE mx.idServicio =
s.idServicio), 0) AS TotalMateriales " +
        "FROM Operativo.Servicio s " +
        "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio " +
        "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente " +
        "WHERE s.idServicio = ?";

```

```

        try (Connection bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
        PreparedStatement stmt = bd.prepareStatement(sql)) {

```

```

            stmt.setInt(1, servicio);
            try (ResultSet rs = stmt.executeQuery())
            {
                TablaConsulta.setModel(model);
                // Agrega los valores a una nueva fila para almacenarlos en el JTable
                while (rs.next()) {
                    model.addRow(new Object[]{
                        rs.getString("ServicioCliente"),
                        rs.getString("fechaInicio"),
                        rs.getString("fechaFinal"),
                        rs.getInt("TotalMateriales")
                    });
                }
            }

```

```

        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null, "Error al cargar los datos: " +
ex.getMessage());
        }
    }

```

```

/**
 * Carga la información de los servicios desde la base de datos y la muestra en un
JComboBox.
 */

```

```

    public void cargarServicios()
    {
        String sql = "SELECT s.idServicio, CONCAT(s.idServicio, ' - ', c.nombreCliente)
AS ServicioCliente " +
            "FROM Operativo.Servicio s " +
            "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio
" +

```

```
"JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente";
```

```
try (Connection bd =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/ServiciosDeMantenimiento", "postgres", "postgres");
    PreparedStatement stmt = bd.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    // Limpiar el JComboBox antes de agregar datos
    cmbConsulta1.removeAllItems();

    // Leer cada servicio del resultado de la consulta
    while (rs.next()) {
        cmbConsulta1.addItem(rs.getString("ServicioCliente"));
    }

} catch (SQLException ex) {
    JOptionPane.showMessageDialog(null, "Error al cargar los servicios: " +
ex.getMessage());
}

private void btnConsultaActionPerformed(java.awt.event.ActionEvent evt) {
    consulta();
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Consulta2().setVisible(true);
        }
    });
}
```